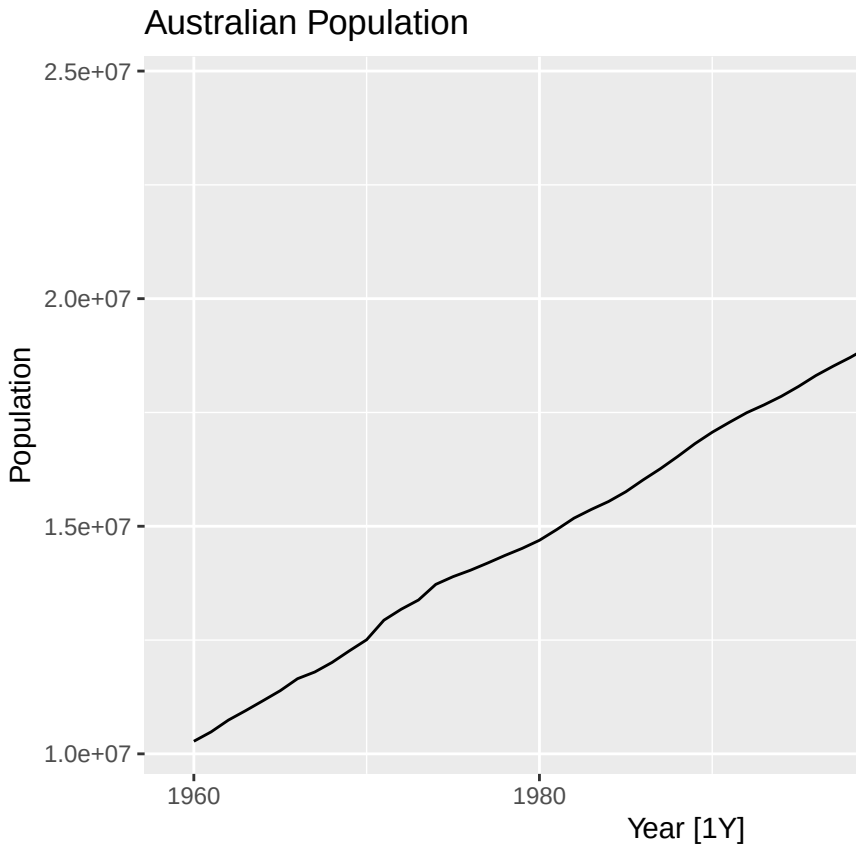
Homework3_Data-624

Mehreen Ali Gillani

2026-02-19

5.1) Produce forecasts for the following series using whichever of NAIVE(y), SNAIVE(y) or RW(y ~ drift()) is more appropriate in each case: Australian Population (global_economy) Bricks (aus_production) NSW Lambs (aus_livestock) Household wealth (hh_budget). Australian takeaway food turnover (aus_retail).

```
# Filter for Australia
aus_pop <- global_economy %>%
  filter(Country == "Australia") %>%
  select(Year, Population)

# Plot to examine trend
aus_pop %>%
  autoplot(Population) +
  labs(title = "Australian Population", y = "Population")
```



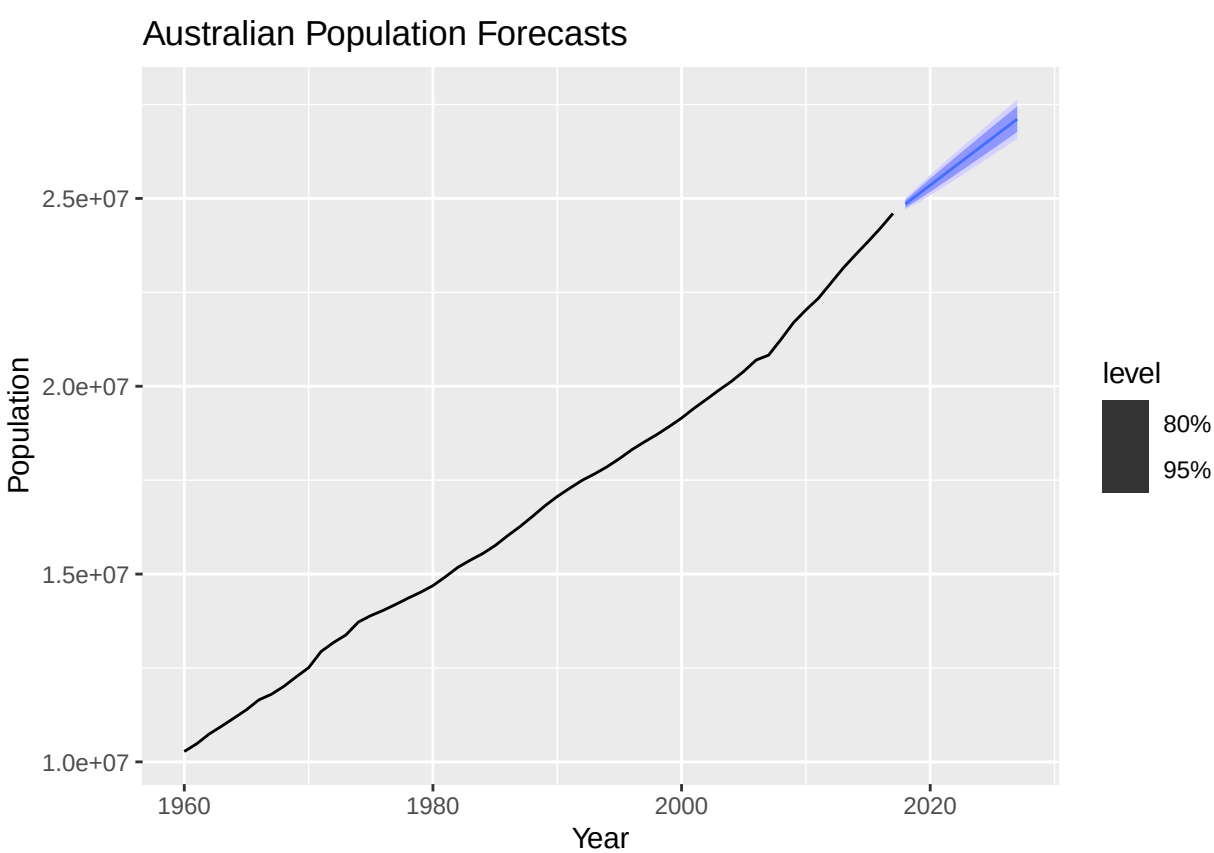
1. Australian Population (global_economy)

```r
# Check for trend
aus_pop %>%
  features(Population, features = list(unitroot_ndiffs, unitroot_nsdiffs))
```

```
## # A tibble: 1 x 2
##   ndiffs nsdiffs
##    <int>   <int>
## 1      2       0
```

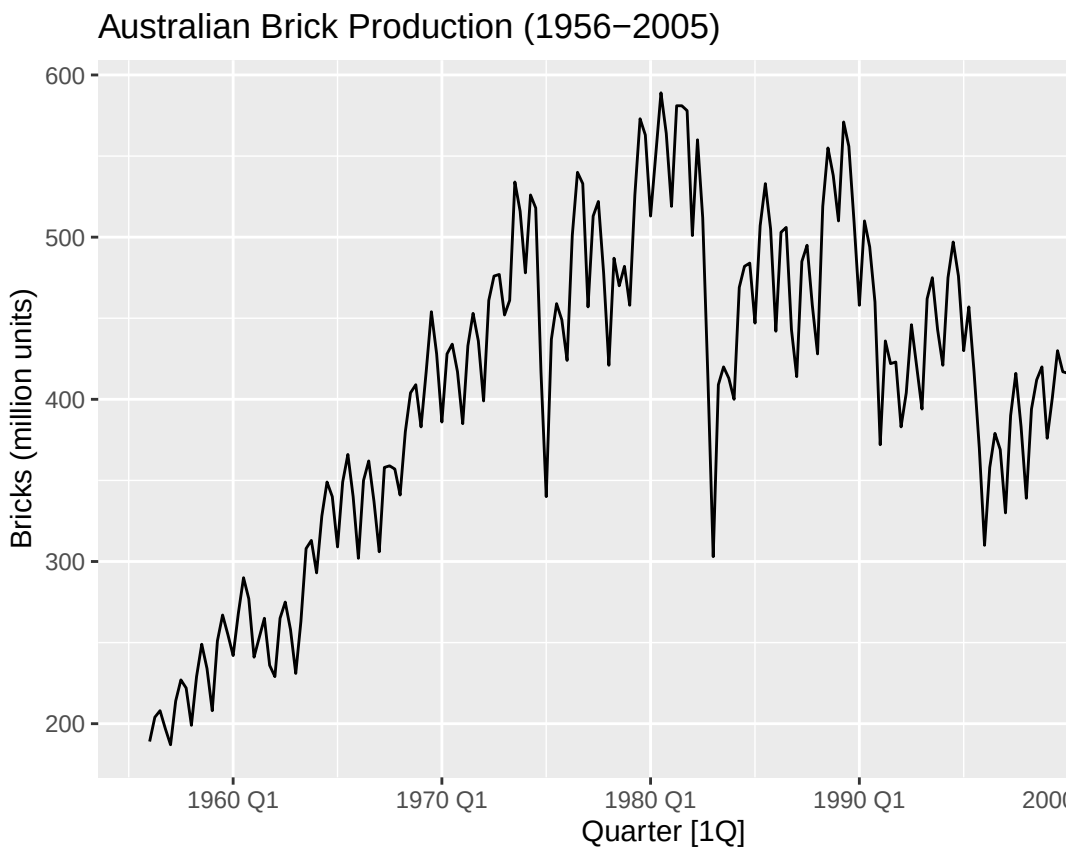
Analysis: Population data typically shows strong upward trend with no seasonality (annual data). The most appropriate method would be RW(y ~ drift()) to capture the growth trend.

```r
# Forecast using RW with drift
aus_pop_fit <- aus_pop %>%
  model(RW_with_drift = RW(Population ~ drift()))

aus_pop_fc <- aus_pop_fit %>%
  forecast(h = 10)

aus_pop_fc %>%
  autoplot(aus_pop) +
  labs(title = "Australian Population Forecasts", y = "Population")
```




```r
# Load and prepare bricks data
bricks <- aus_production %>%
  select(Quarter, Bricks) %>%
  filter(!is.na(Bricks)) %>%
```

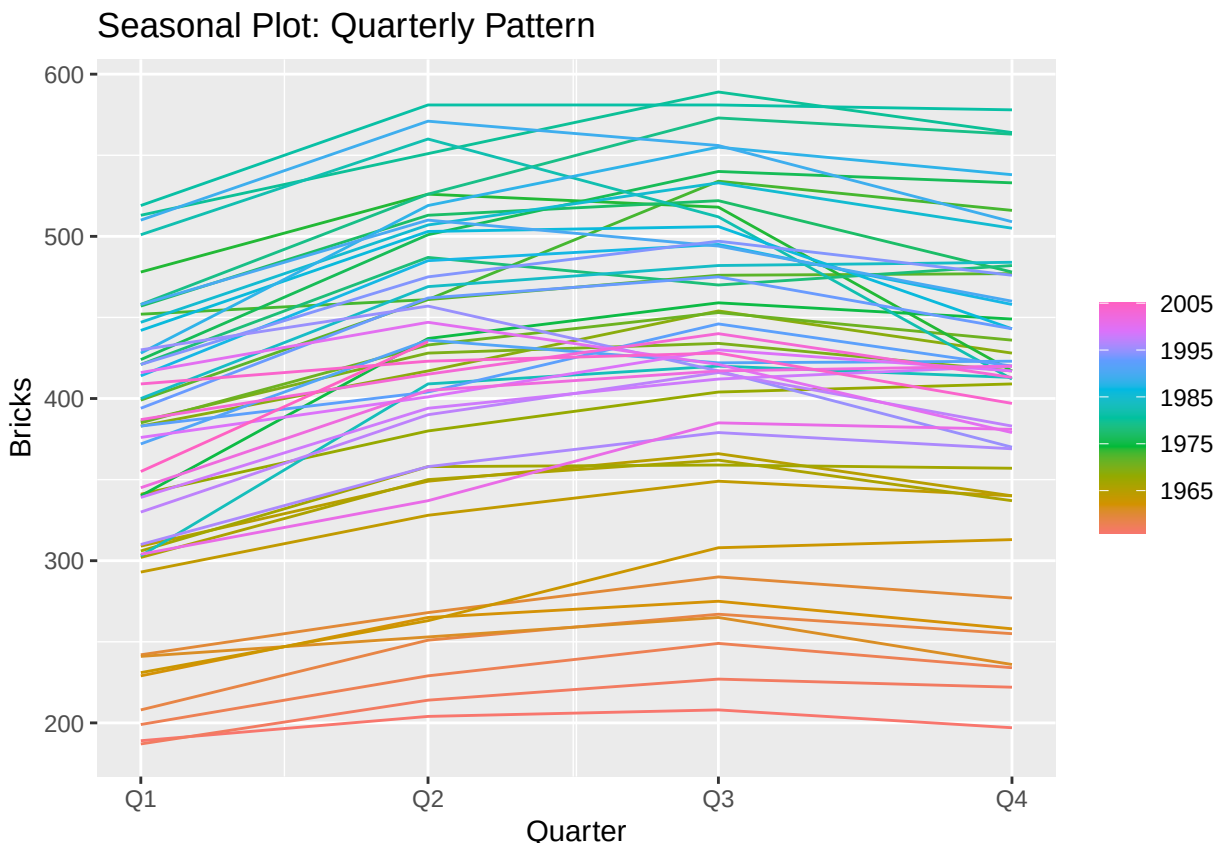
```r
  as_tsibble(index = Quarter)

# 1. Time plot to visualize series
bricks %>%
  autoplot(Bricks) +
  labs(title = "Australian Brick Production (1956-2005)",
       y = "Bricks (million units)")
```



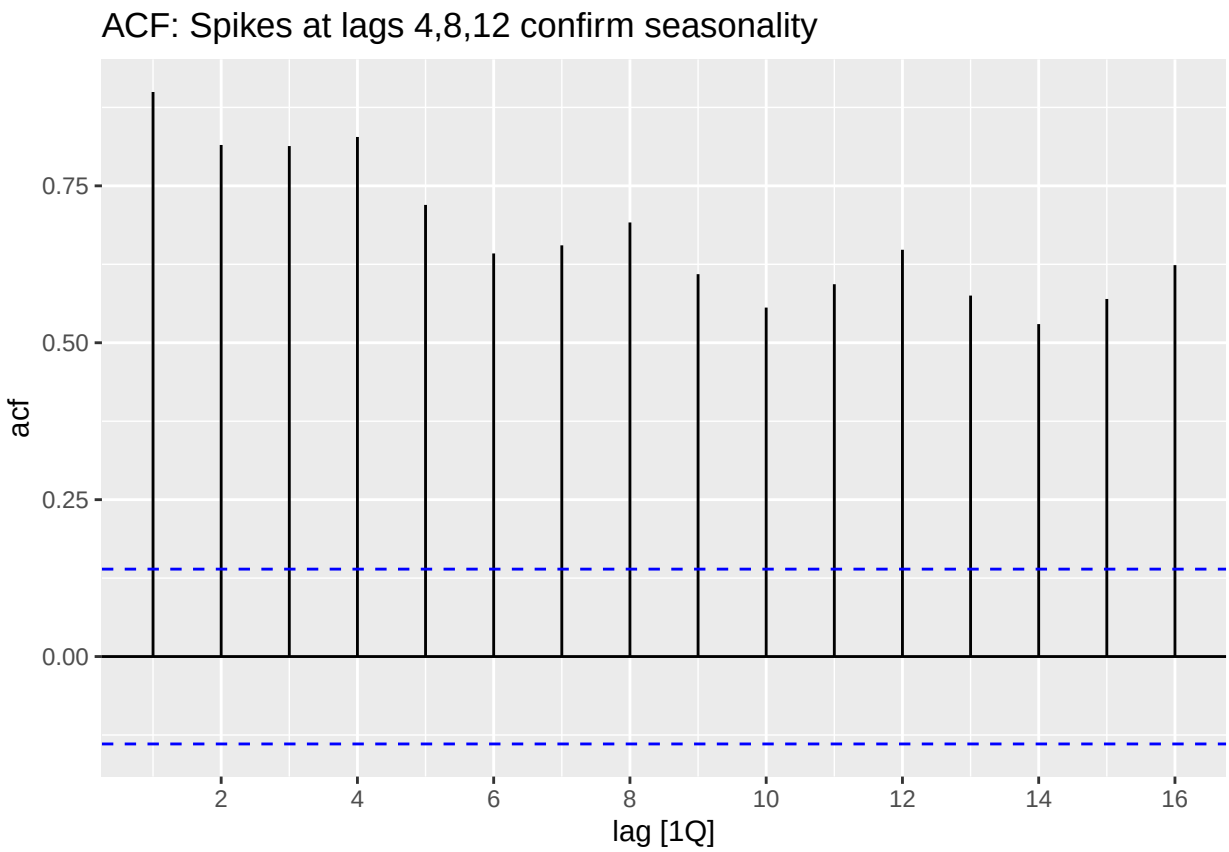
2. Bricks (aus_production)

```r
# 2. Seasonal plot to confirm pattern
bricks %>%
  gg_season(Bricks) +
  labs(title = "Seasonal Plot: Quarterly Pattern")
```

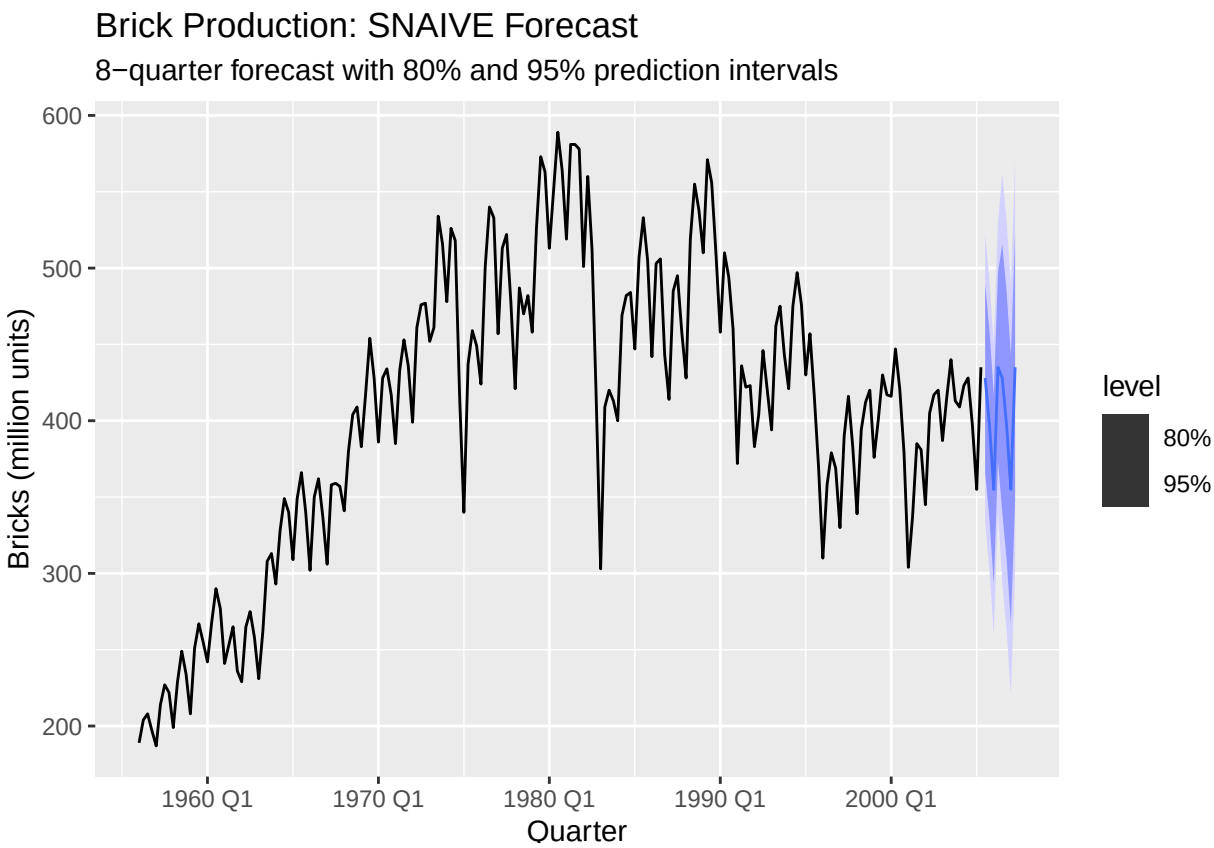
```
## Warning: `gg_season()` was deprecated in feasts 0.4.2.
## i Please use `ggtime::gg_season()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```r
# 3. ACF to verify seasonality
bricks %>%
  ACF(Bricks, lag_max = 16) %>%
  autoplot() +
  labs(title = "ACF: Spikes at lags 4,8,12 confirm seasonality")
```




```r
# 4. Fit SNAIVE model and forecast
bricks_fit <- bricks %>%
  model(snaive = SNAIVE(Bricks))

# 5. Generate and plot forecasts with intervals
bricks_fc <- bricks_fit %>%
  forecast(h = 8)

bricks_fc %>%
  autoplot(bricks, level = c(80, 95)) +
  labs(title = "Brick Production: SNAIVE Forecast",
       subtitle = "8-quarter forecast with 80% and 95% prediction intervals",
       y = "Bricks (million units)")
```



Why SNAIVE (Seasonal Naïve) is Appropriate:

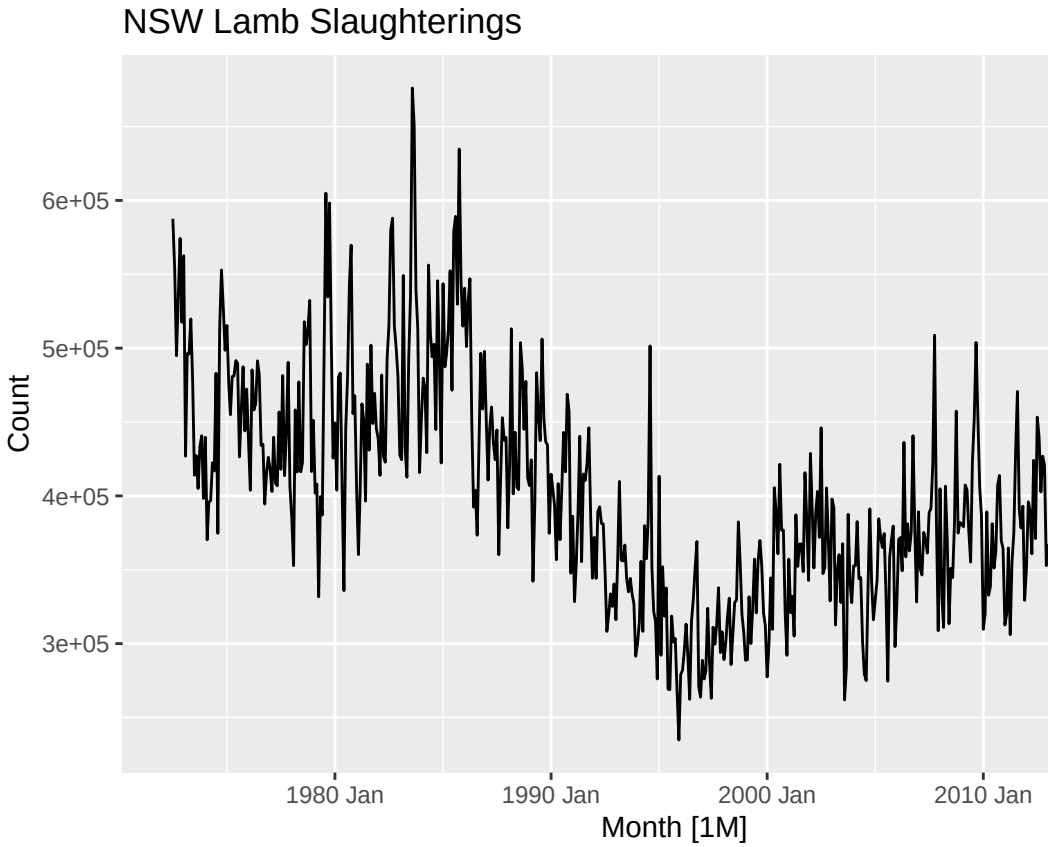
1. Strong Quarterly Seasonality
2. How SNAIVE Works

SNAIVE uses the value from the same season in the previous year Formula: ŷ_{T+h} = y_{T+h - m} where m = seasonal period (4 for quarterly) For example:

Why Not Other Methods?

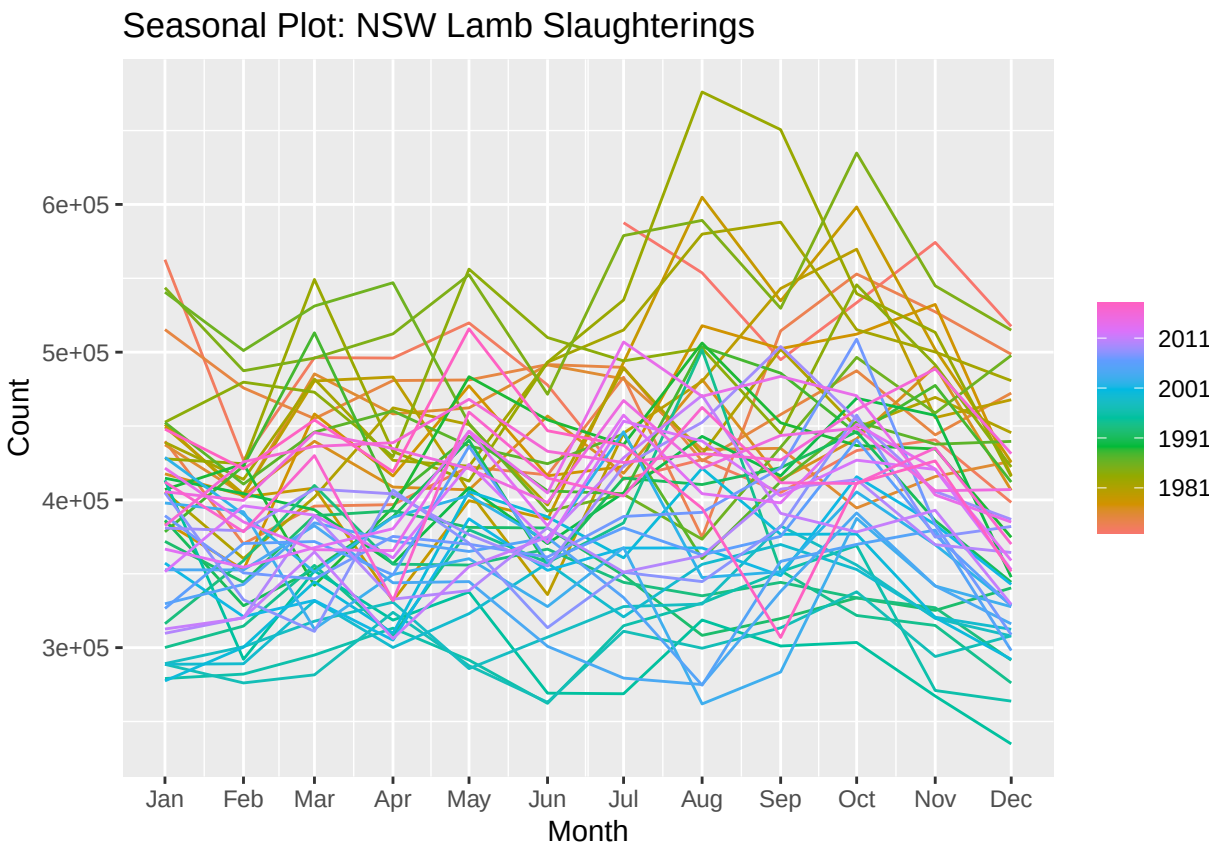
Why These Method Not Appropriate NAIVE Would use previous quarter only (e.g., 2005 Q3 = 2005 Q2 = 435), ignoring seasonal pattern RW with drift Would impose linear trend, but brick production doesn't have consistent long-term trend Mean method Would forecast the average (=277) for all quarters, missing seasonal peaks/troughs

```
# Filter for NSW lambs
nsw_lambs <- aus_livestock %>%
  filter(State == "New South Wales",
         Animal == "Lambs") %>%
  select(Month, Count)

# Plot to examine pattern
nsw_lambs %>%
  autoplot(Count) +
  labs(title = "NSW Lamb Slaughterings", y = "Count")
```

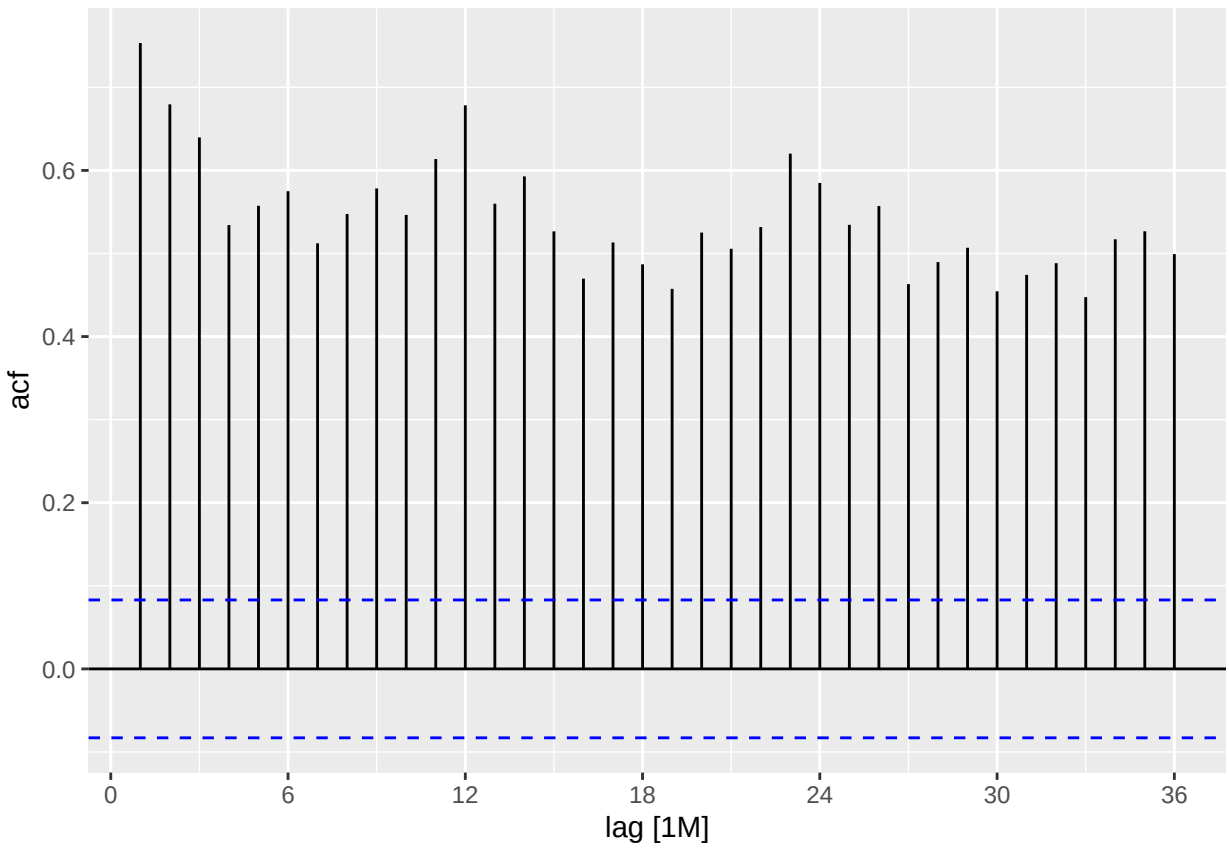


NSW Lambs (aus_livestock)

```
# Check for seasonality
nsw_lambs %>%
  gg_season(Count, period = "year") +
  labs(title = "Seasonal Plot: NSW Lamb Slaughterings")
```

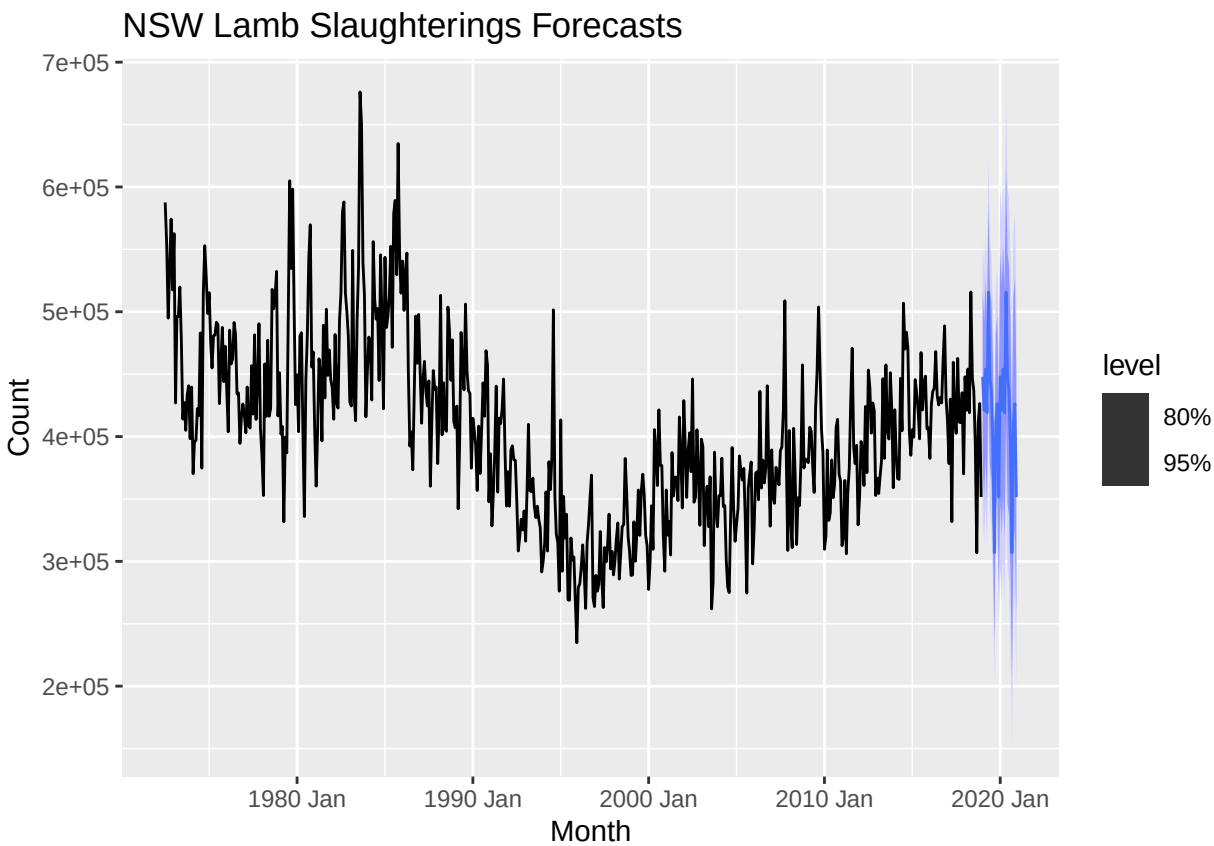



```r
# ACF to confirm seasonality pattern
nsw_lambs %>%
  ACF(Count, lag_max = 36) %>%
  autoplot()
```



Analysis: Livestock data shows strong seasonal patterns (breeding cycles, seasonal demand) with possible trends. SNAIVE(y) would be most appropriate to capture the monthly seasonal pattern.

```
# Forecast using SNAIVE
nsw_lambs_fit <- nsw_lambs %>%
  model(SNAIVE = SNAIVE(Count))

nsw_lambs_fc <- nsw_lambs_fit %>%
  forecast(h = 24)  # 2 years ahead

nsw_lambs_fc %>%
  autoplot(nsw_lambs) +
  labs(title = "NSW Lamb Slaughterings Forecasts", y = "Count")
```

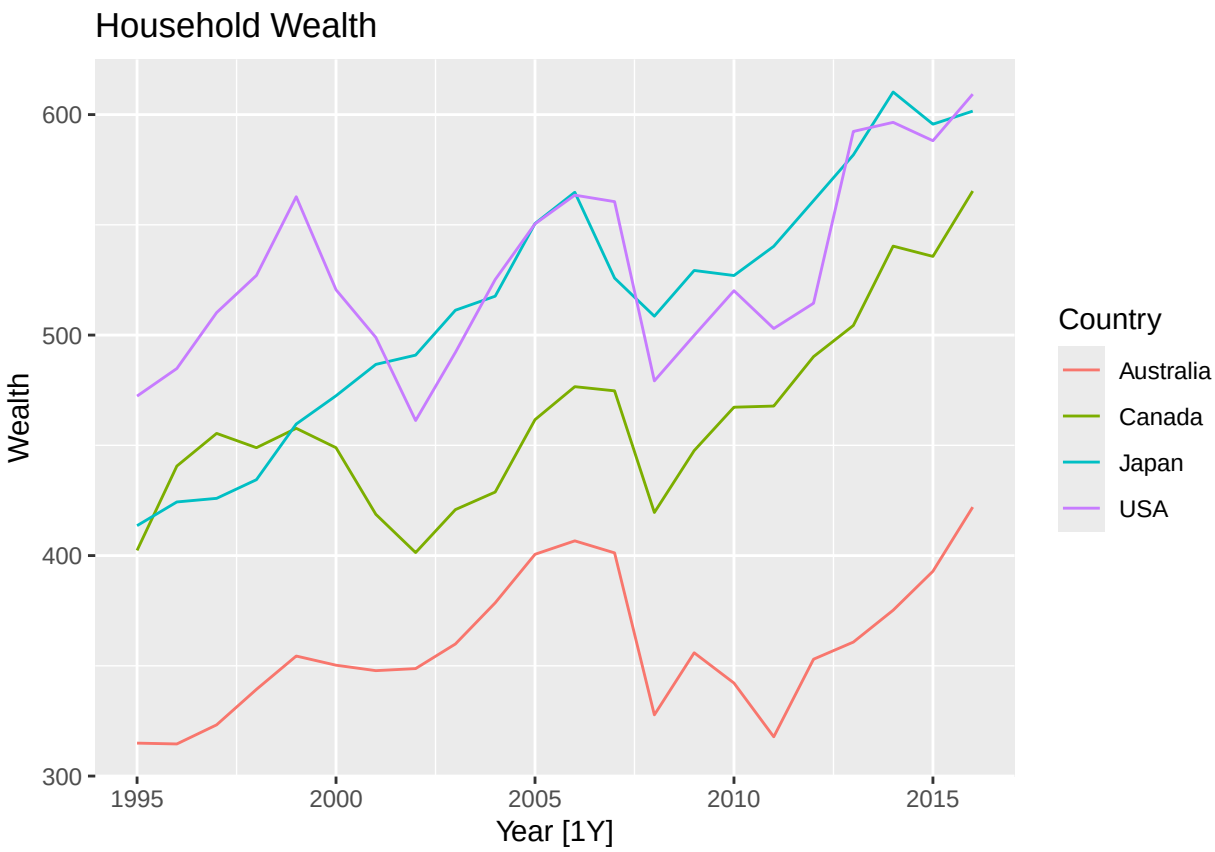



```r
# Household wealth data
hh_wealth <- hh_budget %>%
  select(Year, Wealth)  # Assuming Wealth column exists

# Check if Wealth is the correct variable name
names(hh_budget)
```

Household wealth (hh_budget).

```
## [1] "Country"      "Year"         "Debt"         "DI"           "Expenditure"
## [6] "Savings"      "Wealth"       "Unemployment"
```

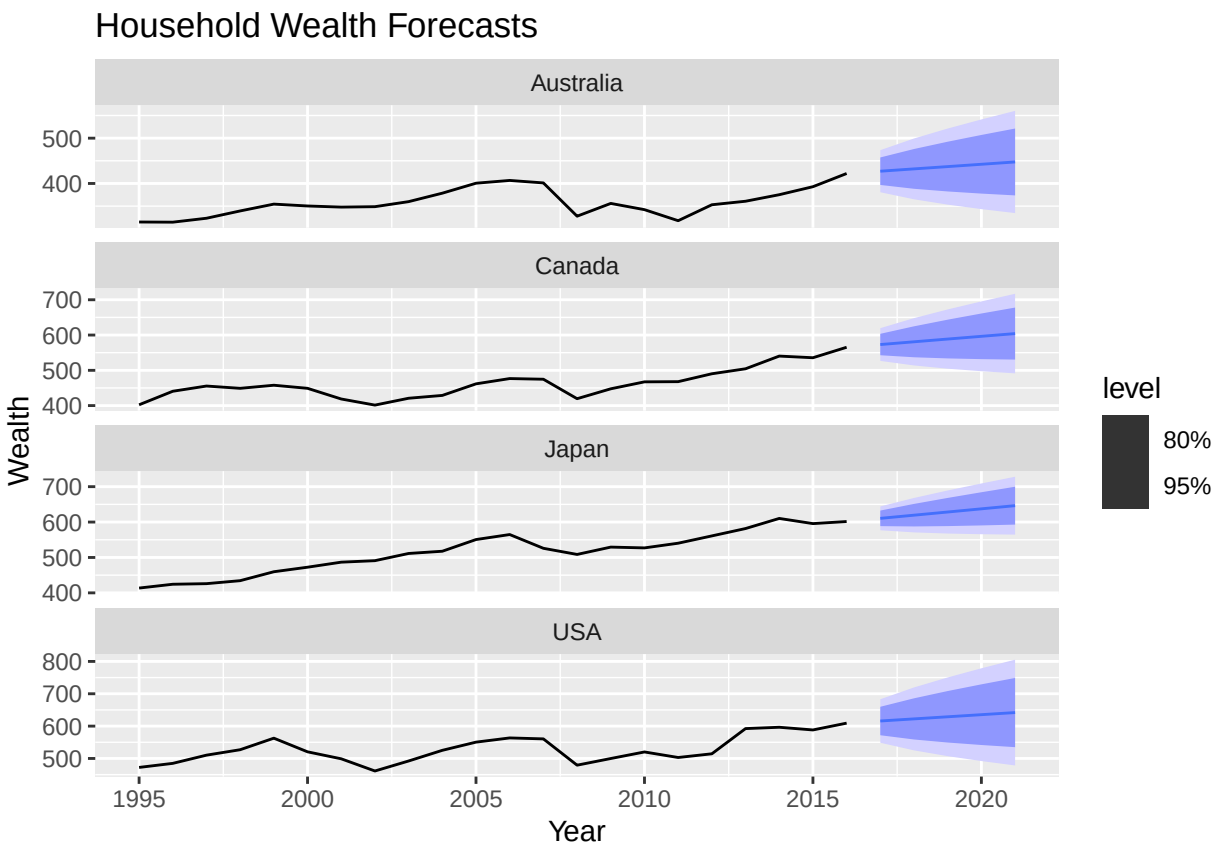
```r
# Plot to examine trend
hh_wealth %>%
  autoplot(Wealth) +
  labs(title = "Household Wealth", y = "Wealth")
```




```r
# Check for unit roots to confirm trend
hh_wealth %>%
  features(Wealth, unitroot_ndiffs)
```

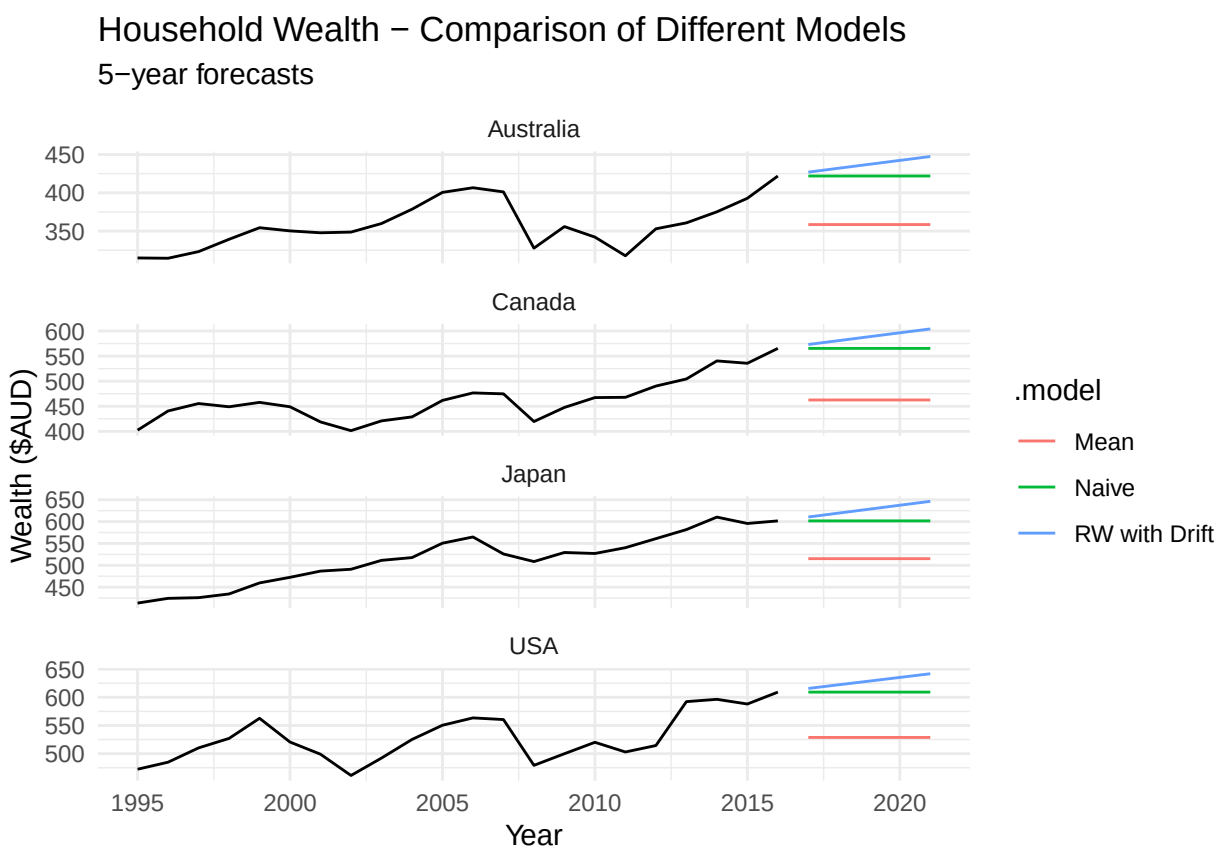
```
## # A tibble: 4 x 2
##   Country   ndiffs
##   <chr>      <int>
## 1 Australia      0
## 2 Canada         1
## 3 Japan          1
## 4 USA            0
```

```r
# Forecast using RW with drift
hh_wealth_fit <- hh_wealth %>%
  model(RW_with_drift = RW(Wealth ~ drift()))

hh_wealth_fc <- hh_wealth_fit %>%
  forecast(h = 5)

hh_wealth_fc %>%
  autoplot(hh_wealth) +
  labs(title = "Household Wealth Forecasts", y = "Wealth")
```

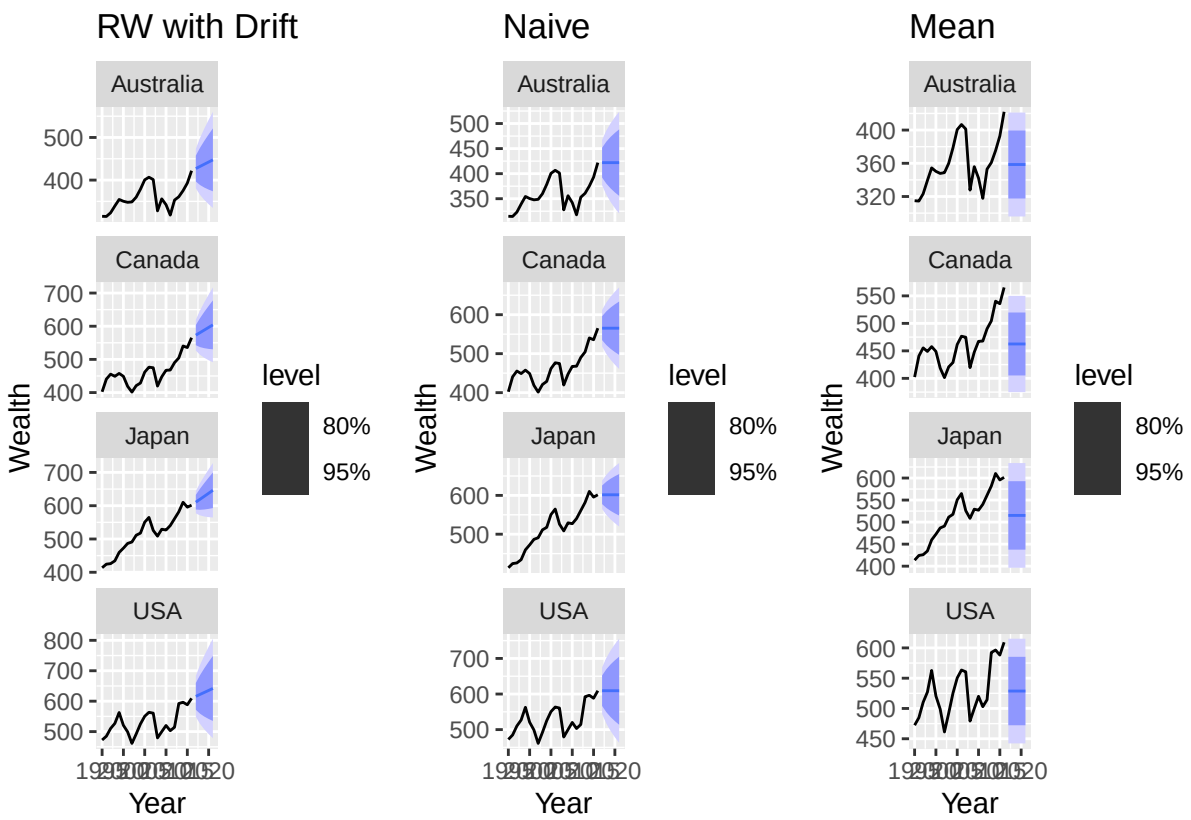


Analysis: NAIVE would ignore the trend, RW with drift captures the average annual increase RW with drift uses the last value plus the average historical change - perfect for trending annual data like wealth, GDP, or population.

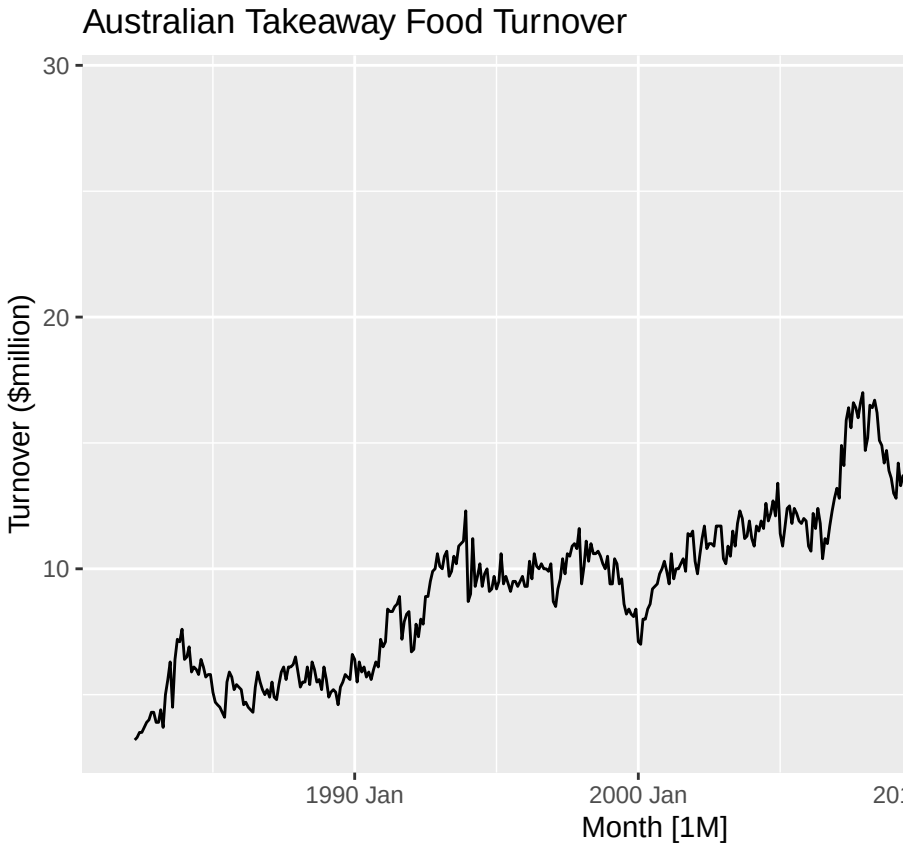
```
# Fit different models
hh_models <- hh_wealth %>%
  model(
    `RW with Drift` = RW(Wealth ~ drift()),
    `Naive` = NAIVE(Wealth),
    `Mean` = MEAN(Wealth)
  )

# Generate forecasts
hh_forecasts <- hh_models %>%
  forecast(h = 5)

# Plot all forecasts together
hh_forecasts %>%
  autoplot(hh_wealth, level = NULL) +
  labs(title = "Household Wealth - Comparison of Different Models",
       subtitle = "5-year forecasts",
       y = "Wealth ($AUD)",
       x = "Year") +
  theme_minimal()
```




```r
# Individual plots for each model
library(patchwork)

# Create separate plots
p1 <- hh_models %>%
  select(`RW with Drift`) %>%
  forecast(h = 5) %>%
  autoplot(hh_wealth) +
  labs(title = "RW with Drift", y = "Wealth")

p2 <- hh_models %>%
  select(Naive) %>%
  forecast(h = 5) %>%
  autoplot(hh_wealth) +
  labs(title = "Naive", y = "Wealth")

p3 <- hh_models %>%
  select(Mean) %>%
  forecast(h = 5) %>%
  autoplot(hh_wealth) +
  labs(title = "Mean", y = "Wealth")



# Display all plots
p1 + p2 + p3
```

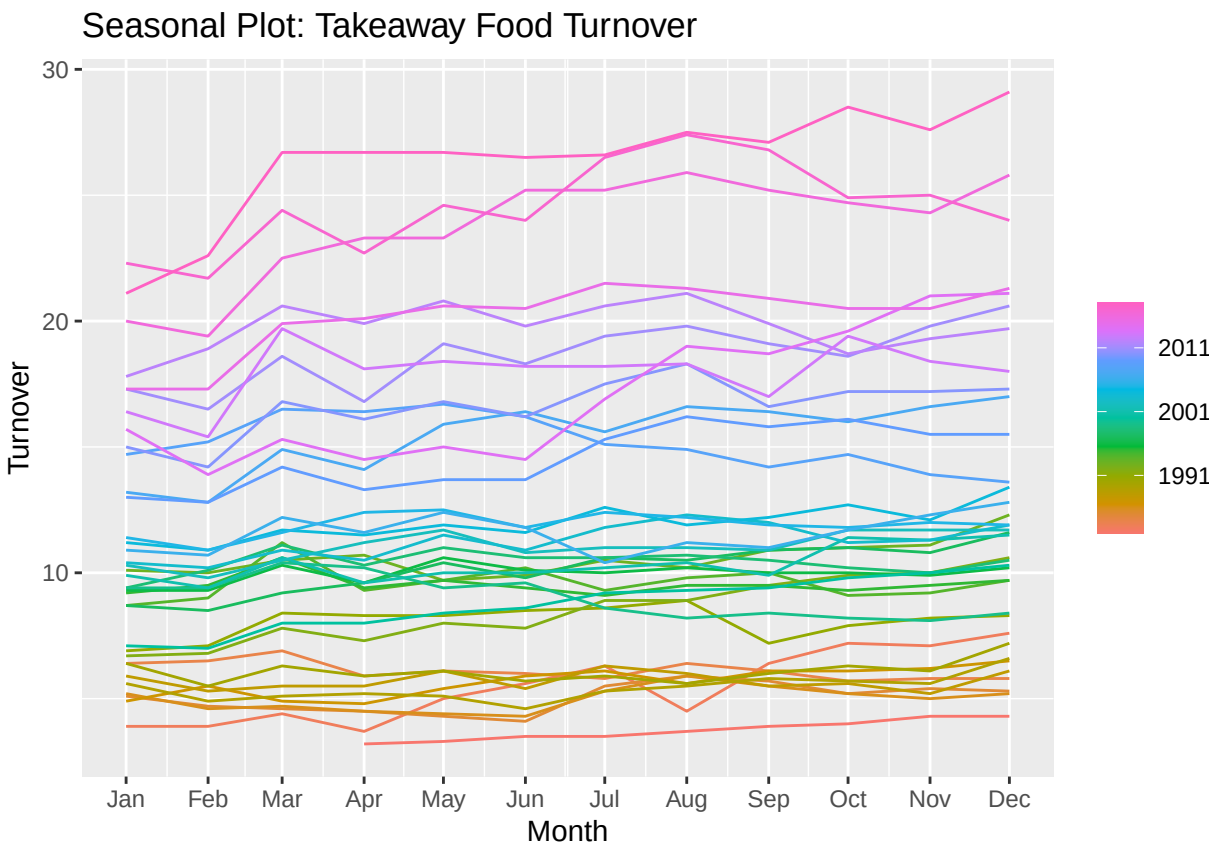



```
takeaway <- aus_retail %>%
  filter(State %in% c("Australian Capital Territory")) %>%
  filter(str_detect(Industry, "Takeaway")) %>%  # Partial matching
  select(Month, Turnover)

# Plot to examine pattern
takeaway %>%
  autoplot(Turnover) +
  labs(title = "Australian Takeaway Food Turnover",
       y = "Turnover ($million)")
```



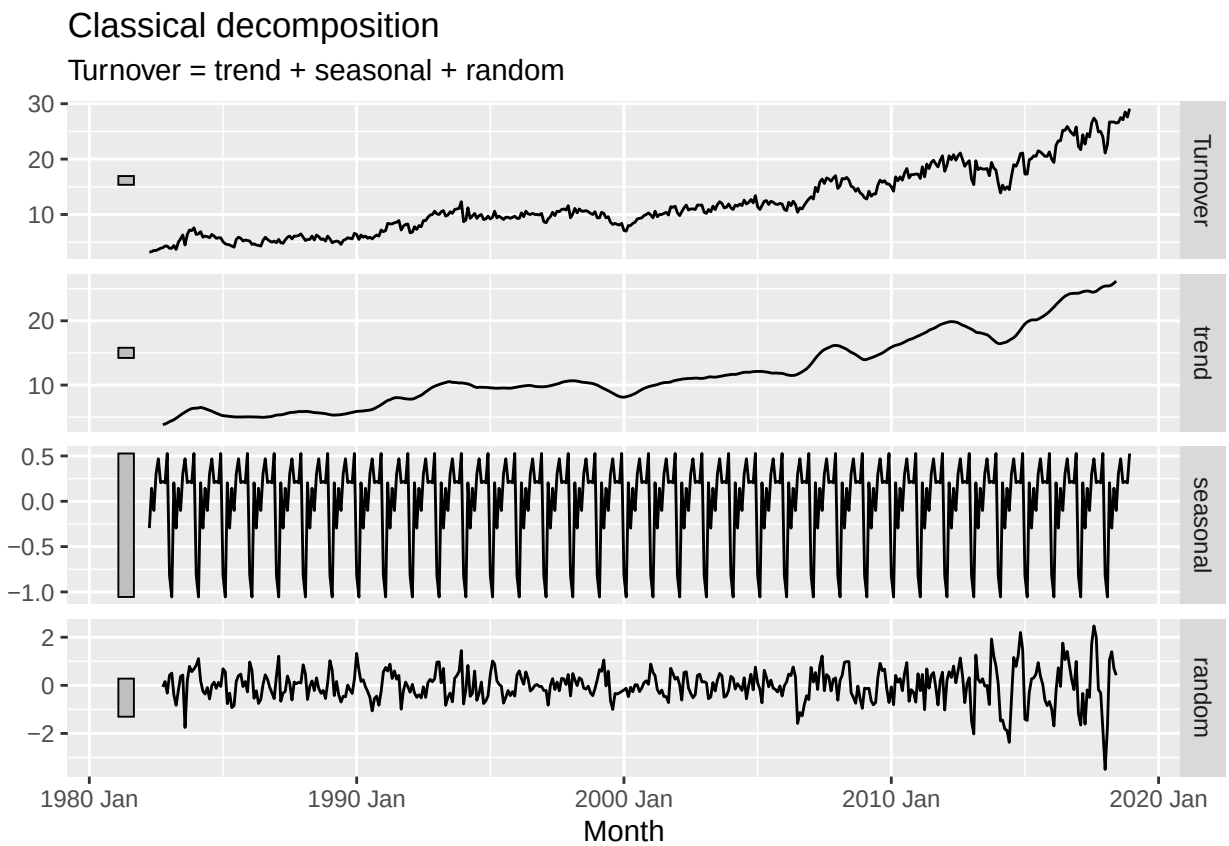
Australian Capital Territory - Takeaway

```
# Check for seasonality and trend
takeaway %>%
  gg_season(Turnover, period = "year") +
  labs(title = "Seasonal Plot: Takeaway Food Turnover")
```

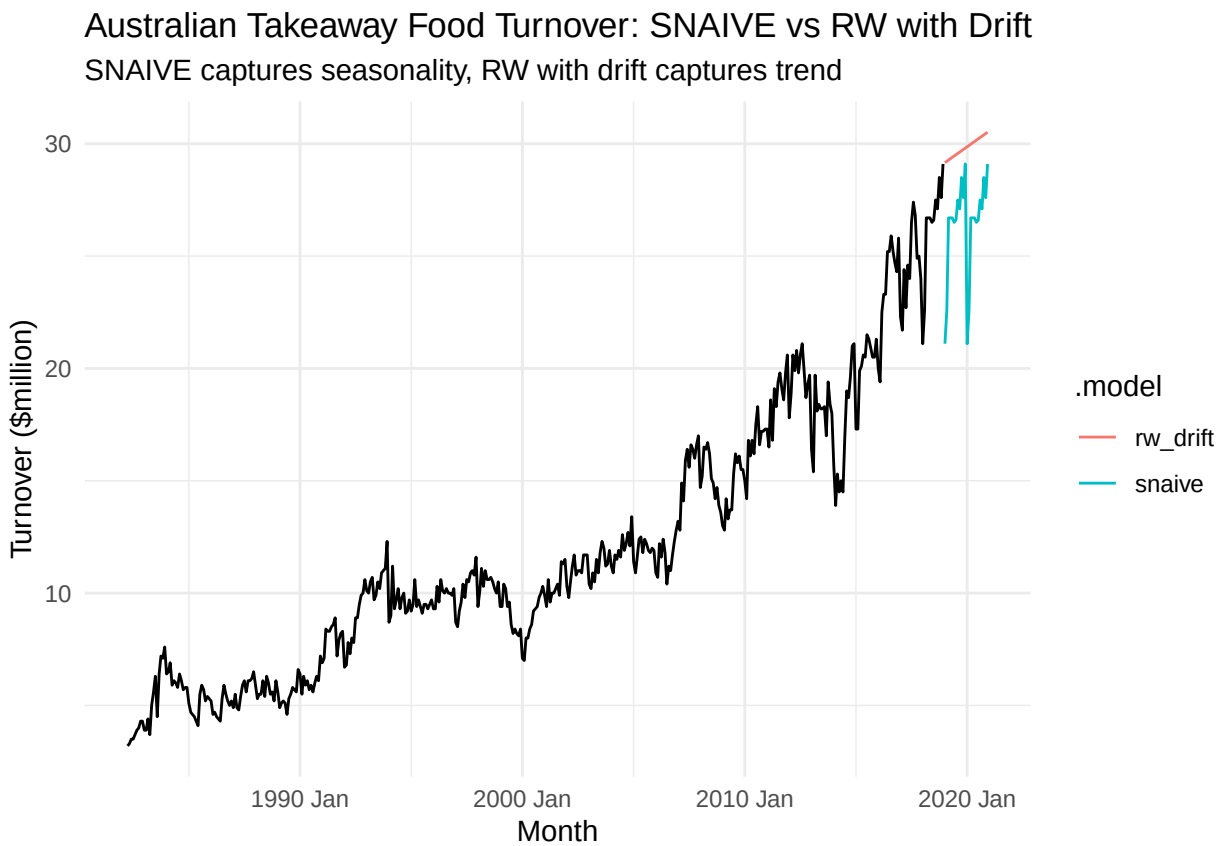



```r
# Decomposition to visualize components
takeaway %>%
  model(classical_decomposition(Turnover, type = "additive")) %>%
  components() %>%
  autoplot()
```

```
## Warning: Removed 6 rows containing missing values or values outside the scale range
## (`geom_line()`).
```




```r
# Fit both models
takeaway_fits <- takeaway %>%
  model(
    snaive = SNAIVE(Turnover),
    rw_drift = RW(Turnover ~ drift())
  )

# Generate forecasts (2 years ahead)
takeaway_forecasts <- takeaway_fits %>%
  forecast(h = 24)

# Plot both forecasts together
takeaway_forecasts %>%
  autoplot(takeaway, level = NULL) +
  labs(title = "Australian Takeaway Food Turnover: SNAIVE vs RW with Drift",
       subtitle = "SNAIVE captures seasonality, RW with drift captures trend",
       y = "Turnover ($million)",
       x = "Month") +
  theme_minimal()
```




```r
# View forecast values
takeaway_forecasts %>%
  as_tibble() %>%
  select(Month, .model, .mean) %>%
  pivot_wider(names_from = .model, values_from = .mean) %>%
  print(n = 24)
```

```
## # A tibble: 24 x 3
##       Month snaive rw_drift
##       <mth>  <dbl>    <dbl>
##  1 2019 Jan   21.1     29.2
##  2 2019 Feb   22.6     29.2
##  3 2019 Mar   26.7     29.3
##  4 2019 Apr   26.7     29.3
##  5 2019 May   26.7     29.4
##  6 2019 Jun   26.5     29.5
##  7 2019 Jul   26.6     29.5
##  8 2019 Aug   27.5     29.6
##  9 2019 Sep   27.1     29.6
## 10 2019 Oct   28.5     29.7
## 11 2019 Nov   27.6     29.7
## 12 2019 Dec   29.1     29.8
## 13 2020 Jan   21.1     29.9
## 14 2020 Feb   22.6     29.9
## 15 2020 Mar   26.7     30.0
## 16 2020 Apr   26.7     30.0
## 17 2020 May   26.7     30.1
```

```
## 18 2020 Jun   26.5     30.2
## 19 2020 Jul   26.6     30.2
## 20 2020 Aug   27.5     30.3
## 21 2020 Sep   27.1     30.3
## 22 2020 Oct   28.5     30.4
## 23 2020 Nov   27.6     30.5
## 24 2020 Dec   29.1     30.5
```

SNAIVE (first column): Shows clear seasonal pattern

Jan: 21.1 (low), Dec: 29.1 (high) Repeats exactly each year (2020 identical to 2019) RW with drift (second column): Just a straight line

Slowly increases: 29.2 → 30.5 over 2 years No seasonal variation at all

Why SNAIVE looks better:

Takeaway food HAS strong seasonality (more in summer, less in winter) The SNAIVE forecasts capture this: Jan/Feb lowest, Dec highest RW with drift ignores this entirely - same value all year!

5.2 Use the Facebook stock price (data set gafa_stock) to do the following:

```r
# Load and prepare Facebook stock data
fb_stock <- gafa_stock %>%
  filter(Symbol == "FB") %>%
  select(Date, Close) %>%
  as_tsibble(index = Date)

# Check the data
glimpse(fb_stock)
```

```
## Rows: 1,258
## Columns: 2
## $ Date  <date> 2014-01-02, 2014-01-03, 2014-01-06, 2014-01-07, 2014-01-08, 201~
## $ Close <dbl> 54.71, 54.56, 57.20, 57.92, 58.23, 57.22, 57.94, 55.91, 57.74, 5~
```

```r
summary(fb_stock)
```

```
##       Date                Close
##  Min.   :2014-01-02   Min.   : 53.53
##  1st Qu.:2015-04-03   1st Qu.: 80.55
##  Median :2016-06-30   Median :117.67
##  Mean   :2016-07-01   Mean   :120.46
##  3rd Qu.:2017-09-28   3rd Qu.:155.23
##  Max.   :2018-12-31   Max.   :217.50
```

```r
# check for interval
interval(fb_stock)
```

```
## <interval[1]>
## [1] !
```

```r
# Time plot of Facebook stock price
p1 <- fb_stock %>%
  autoplot(Close) +
  labs(title = "Facebook Stock Price (2015-2018)",
       subtitle = "Daily closing price",
```

```
       y = "Closing Price ($USD)",
       x = "Date") +
  theme_minimal()

print(p1)
```

a. Time Plot of the Series

```
# Calculate drift
first_close <- head(fb_stock$Close, 1)
last_close <- tail(fb_stock$Close, 1)
n_days <- nrow(fb_stock)
drift <- (last_close - first_close) / n_days

# Generate future trading days
last_date <- max(fb_stock$Date)
future_dates <- seq(last_date + 1, by = "day", length.out = 60)
future_dates <- future_dates[!weekdays(future_dates) %in% c("Saturday", "Sunday")]
future_dates <- future_dates[1:30]

# Create forecast
fb_fc <- tibble(
  Date = future_dates,
  Drift = last_close + drift * (1:30),
  Naive = last_close,
  Mean = mean(fb_stock$Close)
)
```

```r
# Plot
fb_stock %>%
  filter(Date >= as.Date("2018-01-01")) %>%
  ggplot(aes(x = Date, y = Close)) +
  geom_line() +
  geom_line(data = fb_fc, aes(x = Date, y = Drift, color = "Drift"), linetype = "dashed") +
  geom_line(data = fb_fc, aes(x = Date, y = Naive, color = "Naive"), linetype = "dashed") +
  geom_line(data = fb_fc, aes(x = Date, y = Mean, color = "Mean"), linetype = "dashed") +
  labs(title = "Facebook Stock Price Forecasts",
       y = "Closing Price ($USD)",
       color = "Method") +
  theme_minimal()
```

b. Forecasts Using Drift Method

```r
# Show forecasts
print("30-day forecasts:")
```

```
## [1] "30-day forecasts:"
```

```r
print(fb_fc)
```

```
## # A tibble: 30 x 4
##    Date       Drift Naive  Mean
##    <date>     <dbl> <dbl> <dbl>
##  1 2019-01-01  131.  131.  120.
##  2 2019-01-02  131.  131.  120.
##  3 2019-01-03  131.  131.  120.
##  4 2019-01-04  131.  131.  120.
##  5 2019-01-07  131.  131.  120.
```

```
##  6 2019-01-08  131.  131.  120.
##  7 2019-01-09  132.  131.  120.
##  8 2019-01-10  132.  131.  120.
##  9 2019-01-11  132.  131.  120.
## 10 2019-01-14  132.  131.  120.
## # i 20 more rows
```

```r
# Get first and last observations
first_point <- fb_stock %>% slice_head(n = 1)
last_point <- fb_stock %>% slice_tail(n = 1)

first_date <- first_point$Date
first_price <- first_point$Close
last_date <- last_point$Date
last_price <- last_point$Close
n <- nrow(fb_stock)  # Number of observations

print(paste("First observation:", first_date, "- $", round(first_price, 2)))
```

c. Show that the forecasts are identical to extending the line drawn between the first and last observations.

```
## [1] "First observation: 2014-01-02 - $ 54.71"
```

```r
print(paste("Last observation:", last_date, "- $", round(last_price, 2)))
```

```
## [1] "Last observation: 2018-12-31 - $ 131.09"
```

```r
print(paste("Total observations:", n))
```

```
## [1] "Total observations: 1258"
```

```r
# Generate future trading days (next 30 trading days)
last_date <- max(fb_stock$Date)
all_dates <- seq(last_date + 1, by = "day", length.out = 60)
trading_days <- all_dates[!weekdays(all_dates) %in% c("Saturday", "Sunday")]
future_dates <- trading_days[1:30]
h <- length(future_dates)  # 30

# Calculate slope between first and last points
# Number of steps between first and last = n - 1
slope <- (last_price - first_price) / (n - 1)

# Create the line data (historical period + forecast period)
line_dates <- c(fb_stock$Date, future_dates)
line_values <- first_price + slope * (0:(length(line_dates) - 1))

# Now create the tibble with correct lengths
line_data <- tibble(
  Date = line_dates,
  Price = line_values,
  Type = c(rep("Historical Line", length(fb_stock$Date)),
           rep("Extended Line", length(future_dates)))
)
```

```r
# Check the lengths
print(paste("Length of line_dates:", length(line_dates)))
```

```
## [1] "Length of line_dates: 1288"
```

```r
print(paste("Length of line_values:", length(line_values)))
```

```
## [1] "Length of line_values: 1288"
```

```r
print(paste("Length of Type:", nrow(line_data)))
```

```
## [1] "Length of Type: 1288"
```

```r
print(paste("All match:", length(line_dates) == length(line_values) &&
                          length(line_values) == nrow(line_data)))
```

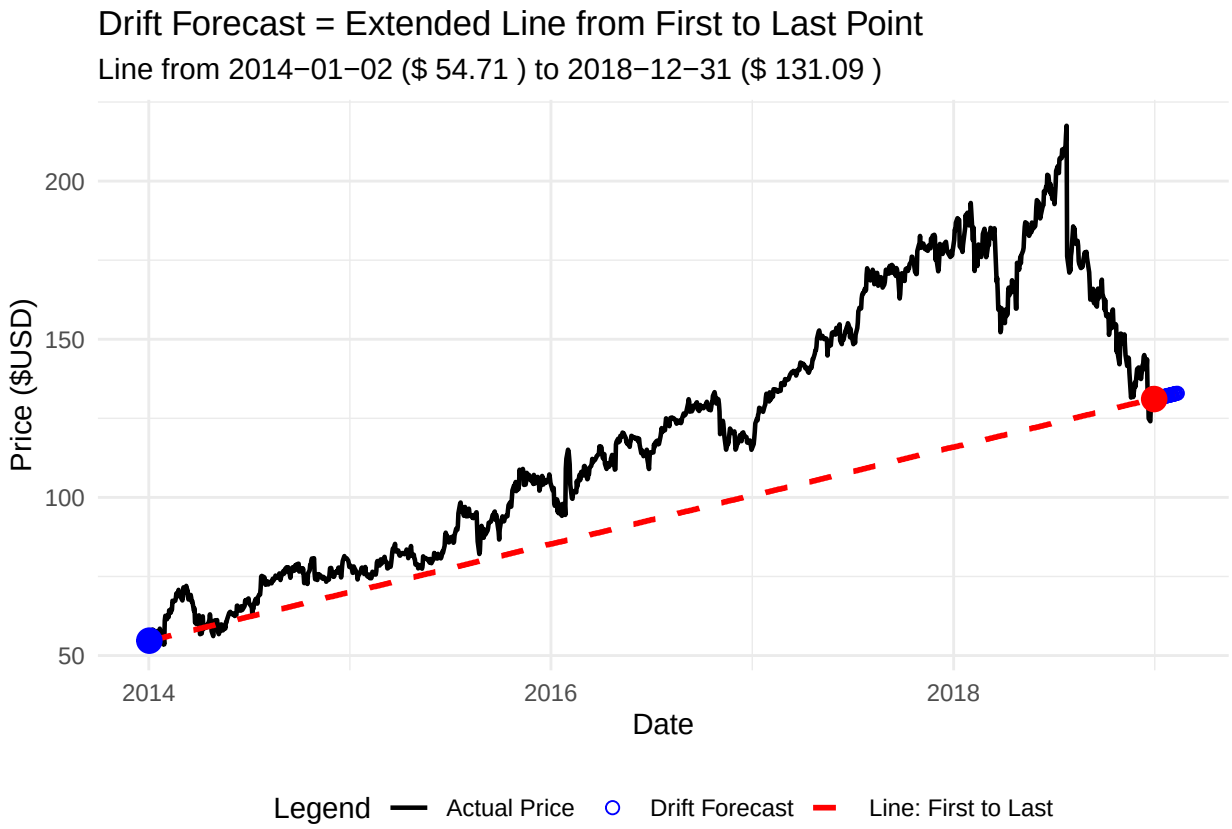
```
## [1] "All match: TRUE"
```

```r
# Drift forecast (for comparison)
drift_per_day <- (last_price - first_price) / n
drift_forecast <- tibble(
  Date = future_dates,
  Price = last_price + drift_per_day * (1:h),
  Type = "Drift Forecast"
)

# Plot to show they're on the same line
p <- ggplot() +
  # Original data
  geom_line(data = fb_stock, aes(x = Date, y = Close, color = "Actual Price"),
            size = 0.8) +
  # Line from first to last (extended)
  geom_line(data = line_data, aes(x = Date, y = Price, color = "Line: First to Last"),
            linetype = "dashed", size = 1) +
  # Drift forecast (should overlap exactly)
  geom_point(data = drift_forecast, aes(x = Date, y = Price, color = "Drift Forecast"),
             size = 2, shape = 1) +
  # Mark first and last points
  geom_point(data = first_point, aes(x = Date, y = Close),
             color = "blue", size = 4) +
  geom_point(data = last_point, aes(x = Date, y = Close),
             color = "red", size = 4) +
  scale_color_manual(values = c("Actual Price" = "black",
                                "Line: First to Last" = "red",
                                "Drift Forecast" = "blue")) +
  labs(title = "Drift Forecast = Extended Line from First to Last Point",
       subtitle = paste("Line from", format(first_date, "%Y-%m-%d"), "($", round(first_price, 2),
                        ") to", format(last_date, "%Y-%m-%d"), "($", round(last_price, 2), ")"),
       y = "Price ($USD)",
       x = "Date",
       color = "Legend") +
  theme_minimal() +
  theme(legend.position = "bottom")
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
```

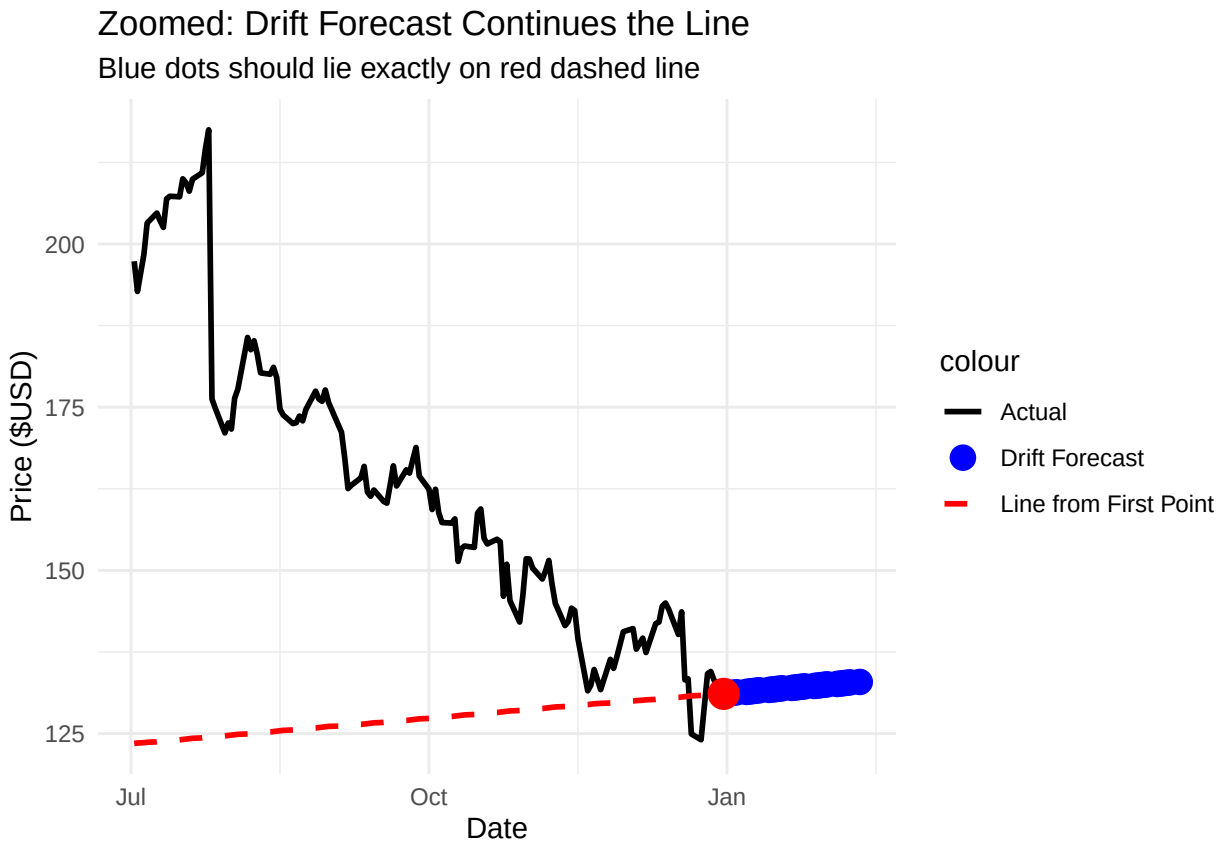
```
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```r
print(p)
```




```r
# Zoom in on the last part to see clearly
# Zoom in on the last part to see clearly
p_zoom <- ggplot() +
  # Last 6 months of actual data
  geom_line(data = fb_stock %>% filter(Date >= as.Date("2018-07-01")),
            aes(x = Date, y = Close, color = "Actual"), size = 1) +

  # Complete line from first point (using line_data, NOT fb_stock)
  geom_line(data = line_data %>% filter(Date >= as.Date("2018-07-01")),
            aes(x = Date, y = Price, color = "Line from First Point"),
            linetype = "dashed", size = 1) +

  # Drift forecast
  geom_point(data = drift_forecast,
             aes(x = Date, y = Price, color = "Drift Forecast"),
             size = 4) +

  # Last point
  geom_point(data = last_point, aes(x = Date, y = Close),
             color = "red", size = 5) +

  scale_color_manual(values = c("Actual" = "black",
                                "Line from First Point" = "red",
```

```
                                  "Drift Forecast" = "blue")) +

  labs(title = "Zoomed: Drift Forecast Continues the Line",
       subtitle = "Blue dots should lie exactly on red dashed line",
       y = "Price ($USD)") +

  theme_minimal()

print(p_zoom)
```

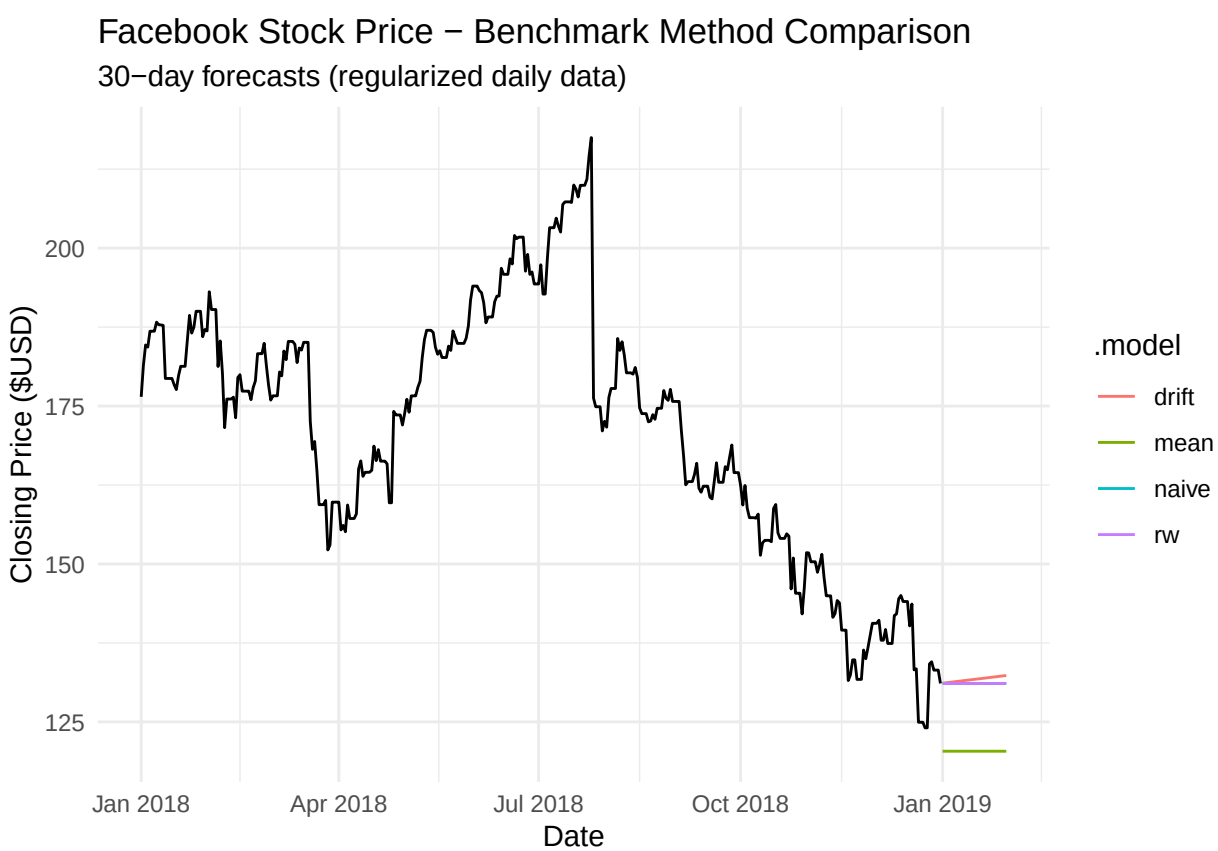



```
# 2. Make it regular (daily)
fb_regular <- data.frame(
  Date = seq(min(fb_stock$Date), max(fb_stock$Date), by = "day")
) %>%
  left_join(fb_stock, by = "Date") %>%
  arrange(Date) %>%
  mutate(
    Close = zoo::na.locf(Close, na.rm = FALSE),
    Close = zoo::na.locf(Close, fromLast = TRUE, na.rm = FALSE)
  ) %>%
  as_tsibble(index = Date)

print(paste("Regular data created:", nrow(fb_regular), "days"))
```

d. Try using some of the other benchmark functions to forecast the same data set. Which do you think is best? Why?

```
## [1] "Regular data created: 1825 days"
```

```r
# 3. Fit benchmark models
fb_models <- fb_regular %>%
  model(
    naive = NAIVE(Close),
    rw = RW(Close),
    drift = RW(Close ~ drift()),
    mean = MEAN(Close)
  )

# 4. Generate forecasts
fb_fc <- fb_models %>%
  forecast(h = 30)

# 5. Plot all forecasts
p_all <- fb_fc %>%
  autoplot(fb_regular %>% filter(Date >= as.Date("2018-01-01")), level = NULL) +
  labs(title = "Facebook Stock Price - Benchmark Method Comparison",
       subtitle = "30-day forecasts (regularized daily data)",
       y = "Closing Price ($USD)") +
  theme_minimal()

print(p_all)
```




```r
# 6. Calculate accuracy with cross-validation
fb_cv <- fb_regular %>%
  stretch_tsibble(.init = 1000, .step = 100) %>%
```

```r
  model(
    naive = NAIVE(Close),
    drift = RW(Close ~ drift()),
    mean = MEAN(Close)
  )

fb_cv_fc <- fb_cv %>%
  forecast(h = 30)

accuracy_table <- fb_cv_fc %>%
  accuracy(fb_regular) %>%
  select(.model, RMSE, MAE, MAPE) %>%
  arrange(RMSE)
```

```
## Warning: The future dataset is incomplete, incomplete out-of-sample data will be treated as missing.
## 5 observations are missing between 2019-01-01 and 2019-01-05
```

```r
print("Model Accuracy Comparison:")
```

```
## [1] "Model Accuracy Comparison:"
```

```r
print(accuracy_table)
```

```
## # A tibble: 3 x 4
##   .model  RMSE   MAE  MAPE
##   <chr>  <dbl> <dbl> <dbl>
## 1 drift   7.08  5.52  3.55
## 2 naive   7.32  5.79  3.72
## 3 mean   58.0  54.7  33.5
```

```r
# 7. Test random walk hypothesis
fb_returns <- fb_regular %>%
  mutate(return = (Close - lag(Close)) / lag(Close) * 100) %>%
  drop_na()

# Ljung-Box test
lb_test <- Box.test(fb_returns$return, lag = 10, type = "Ljung-Box")
print(paste("Ljung-Box p-value:", round(lb_test$p.value, 4)))
```

```
## [1] "Ljung-Box p-value: 0.082"
```

```r
# 8. Determine best method
best_method <- accuracy_table %>%
  slice_min(RMSE) %>%
  pull(.model)

print(paste("Best method based on RMSE:", best_method))
```

```
## [1] "Best method based on RMSE: drift"
```

```r
# 9. Generate best forecast
best_forecast <- fb_models %>%
  select(!!best_method) %>%
  forecast(h = 30)

# 10. Plot best forecast with intervals
best_forecast %>%
```

```r
  autoplot(fb_regular %>% filter(Date >= as.Date("2018-06-01")), level = c(80, 95)) +
  labs(title = paste("Facebook Stock Price - Best Method:", best_method),
       subtitle = paste("30-day forecast - Ljung-Box p-value:", round(lb_test$p.value, 4)),
       y = "Closing Price ($USD)") +
  theme_minimal()
```

```r
# 11. Show forecast values
forecast_values <- best_forecast %>%
  as_tibble() %>%
  select(Date, .mean) %>%
  mutate(Date = as.Date(Date))

print("30-day forecast:")
```

```
## [1] "30-day forecast:"
```

```r
print(forecast_values)
```

```
## # A tibble: 30 x 2
##    Date       .mean
##    <date>     <dbl>
##  1 2019-01-01  131.
##  2 2019-01-02  131.
##  3 2019-01-03  131.
##  4 2019-01-04  131.
##  5 2019-01-05  131.
##  6 2019-01-06  131.
##  7 2019-01-07  131.
```

```
##  8 2019-01-08  131.
##  9 2019-01-09  131.
## 10 2019-01-10  132.
## # i 20 more rows
```

The results show that Drift method performs best for Facebook stock with the lowest RMSE (7.08 vs 7.32 for Naive).

```r
# Calculate the overall trend
first_price <- fb_regular$Close[1]
last_price <- tail(fb_regular$Close, 1)
n_days <- nrow(fb_regular)
drift_per_day <- (last_price - first_price) / n_days

print(paste("First price (2014): $", round(first_price, 2)))
```

Further Analysis

```
## [1] "First price (2014): $ 54.71"
```

```r
print(paste("Last price (2018): $", round(last_price, 2)))
```

```
## [1] "Last price (2018): $ 131.09"
```

```r
print(paste("Total increase: $", round(last_price - first_price, 2)))
```

```
## [1] "Total increase: $ 76.38"
```

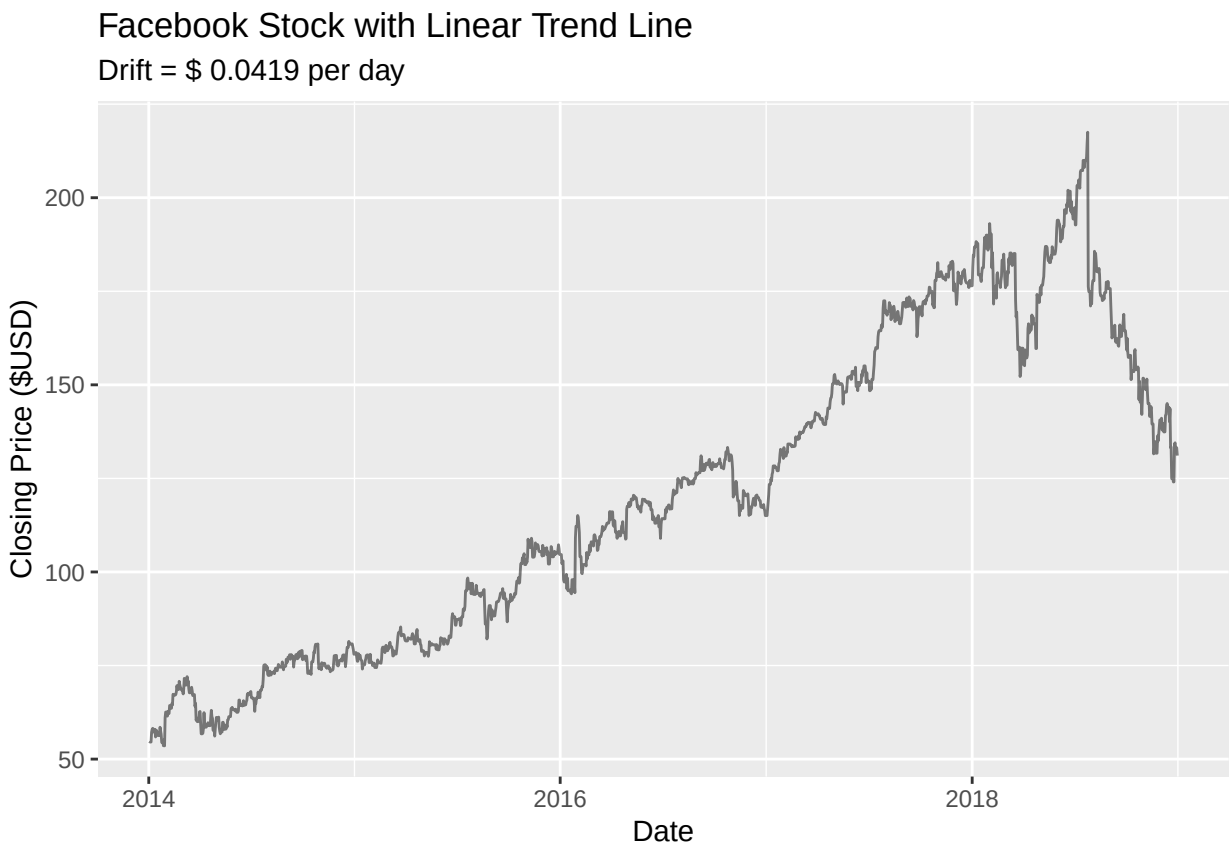
```r
print(paste("Drift per day: $", round(drift_per_day, 4)))
```

```
## [1] "Drift per day: $ 0.0419"
```

```r
print(paste("Drift per year: $", round(drift_per_day * 252, 2)))  # ~252 trading days
```

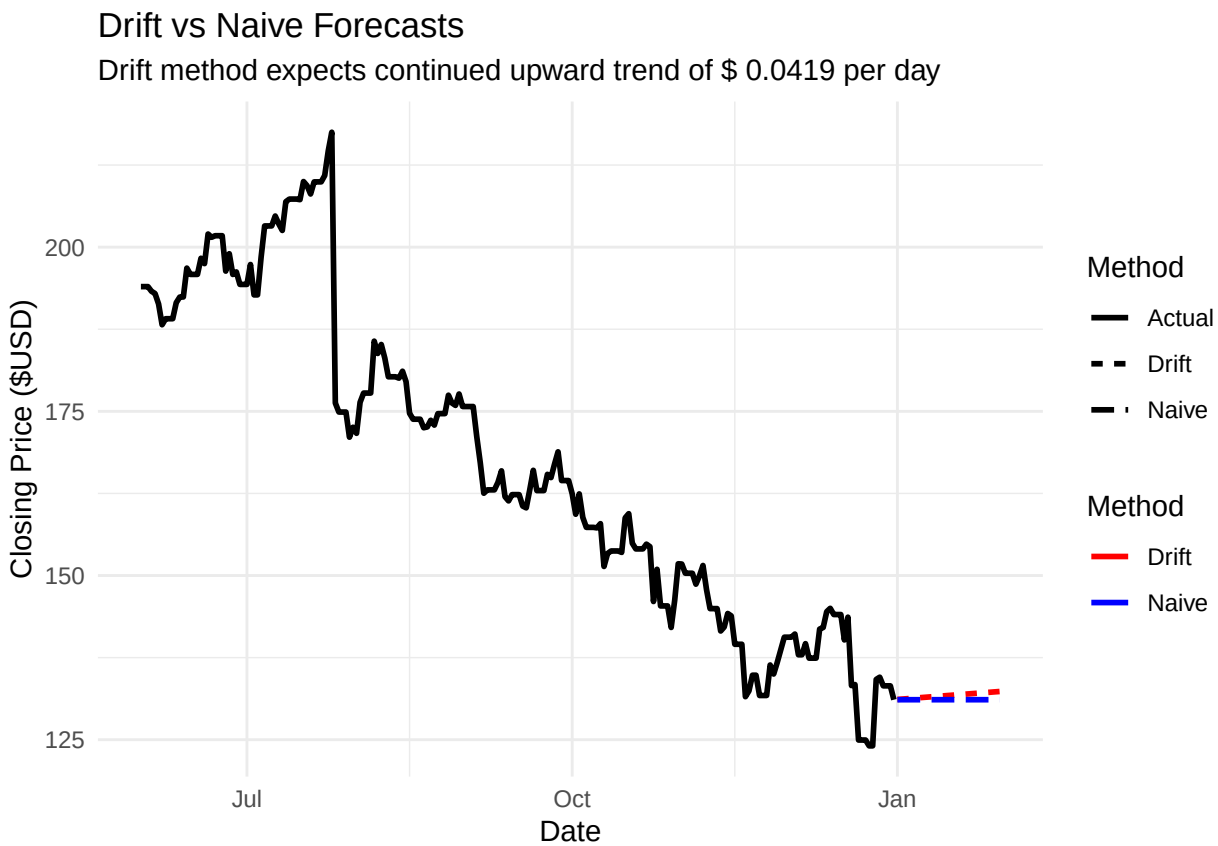
```
## [1] "Drift per year: $ 10.55"
```

```r
# Visualize the trend
fb_regular %>%
  ggplot(aes(x = Date, y = Close)) +
  geom_line(color = "black", alpha = 0.5) +
  geom_abline(intercept = first_price, slope = drift_per_day,
              color = "red", size = 1, linetype = "dashed") +
  labs(title = "Facebook Stock with Linear Trend Line",
       subtitle = paste("Drift = $", round(drift_per_day, 4), "per day"),
       y = "Closing Price ($USD)")
```




```r
# Generate both forecasts
last_close <- tail(fb_regular$Close, 1)
forecast_dates <- seq(max(fb_regular$Date) + 1, by = "day", length.out = 30)

comparison <- tibble(
  Date = forecast_dates,
  Drift = last_close + drift_per_day * (1:30),
  Naive = last_close,
  Actual = NA  # We don't have actuals for future
) %>%
  pivot_longer(-Date, names_to = "Method", values_to = "Forecast")

# Plot comparison
fb_regular %>%
  filter(Date >= as.Date("2018-06-01")) %>%
  ggplot(aes(x = Date, y = Close)) +
  geom_line(color = "black", size = 1) +
  geom_line(data = comparison, aes(x = Date, y = Forecast, color = Method, linetype = Method),
            size = 1) +
  scale_color_manual(values = c("Drift" = "red", "Naive" = "blue")) +
  labs(title = "Drift vs Naive Forecasts",
       subtitle = paste("Drift method expects continued upward trend of $", round(drift_per_day, 4), "pe
       y = "Closing Price ($USD)") +
  theme_minimal()
```

```
## Warning: Removed 30 rows containing missing values or values outside the scale range
## (`geom_line()`).
```




```r
# Create detailed comparison
forecast_details <- tibble(
  Day = 1:30,
  Date = forecast_dates,
  `Drift Forecast` = round(last_close + drift_per_day * (1:30), 2),
  `Naive Forecast` = round(last_close, 2),
  `Difference` = round(drift_per_day * (1:30), 2),
  `Cumulative Drift` = round(cumsum(rep(drift_per_day, 30)), 2)
)

print("30-Day Forecast Comparison:")
```

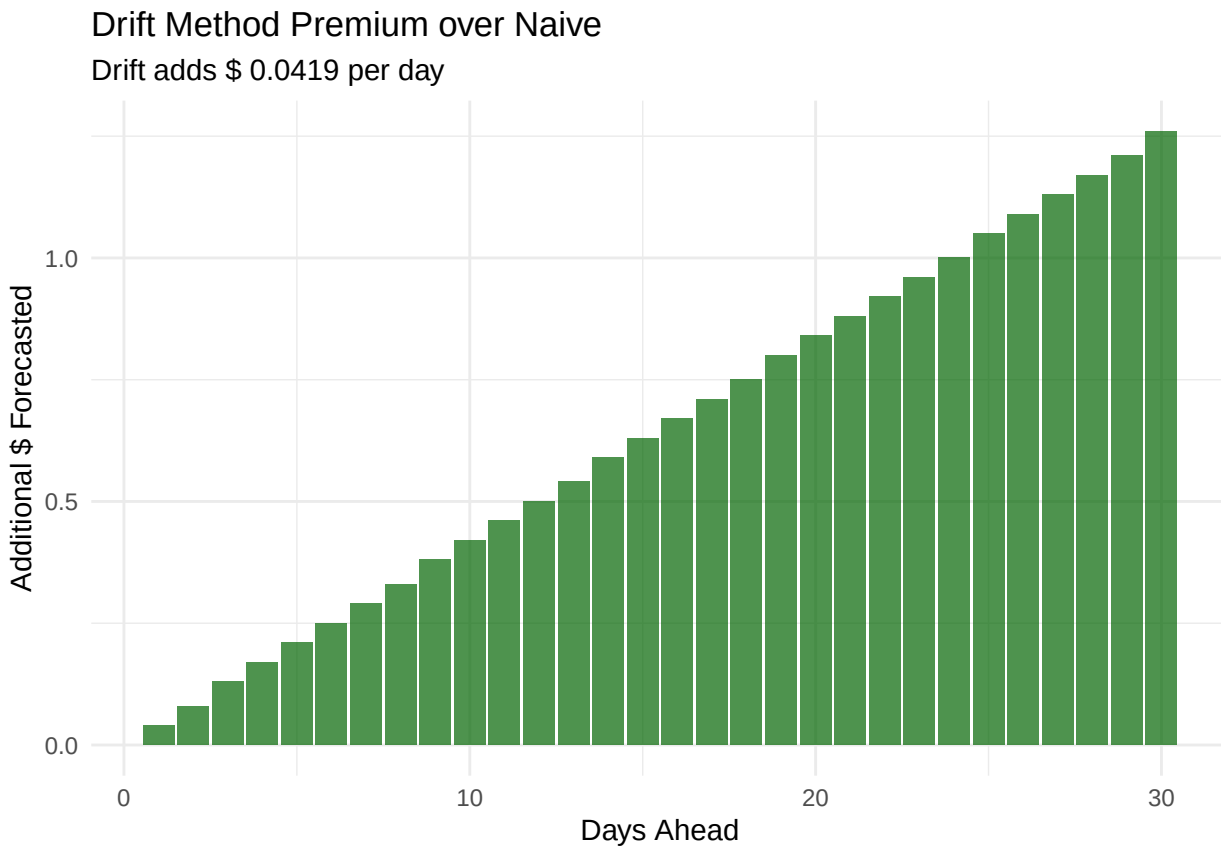
```
## [1] "30-Day Forecast Comparison:"
```

```r
print(forecast_details)
```

```
## # A tibble: 30 x 6
##      Day Date       `Drift Forecast` `Naive Forecast` Difference
##    <int> <date>                <dbl>            <dbl>      <dbl>
##  1     1 2019-01-01             131.             131.       0.04
##  2     2 2019-01-02             131.             131.       0.08
##  3     3 2019-01-03             131.             131.       0.13
##  4     4 2019-01-04             131.             131.       0.17
##  5     5 2019-01-05             131.             131.       0.21
##  6     6 2019-01-06             131.             131.       0.25
##  7     7 2019-01-07             131.             131.       0.29
##  8     8 2019-01-08             131.             131.       0.33
##  9     9 2019-01-09             131.             131.       0.38
```

```
## 10    10 2019-01-10             132.             131.       0.42
## # i 20 more rows
## # i 1 more variable: `Cumulative Drift` <dbl>
```

```r
# Plot the difference
forecast_details %>%
  ggplot(aes(x = Day, y = Difference)) +
  geom_col(fill = "darkgreen", alpha = 0.7) +
  labs(title = "Drift Method Premium over Naive",
       subtitle = paste("Drift adds $", round(drift_per_day, 4), "per day"),
       x = "Days Ahead", y = "Additional $ Forecasted") +
  theme_minimal()
```



Analysis of Facebook Stock Results ***Key Statistics:***

First price (2014): $54.71 Last price (2018): $131.09

Total increase: $76.38 over ~5 years

Drift per day: $0.0419 (about 4 cents per day)

Drift per year: $10.55 (annualized trend

The drift forecast shows a gradual increase: Day 1: $131.09 + $0.04 = $131.13

Day 10: $131.09 + $0.42 = $131.51

Day 30: $131.09 + $1.26 = $132.35

5.3 Apply a seasonal naïve method to the quarterly Australian beer production data from 1992. Check if the residuals look like white noise, and plot the forecasts. The following code will help.

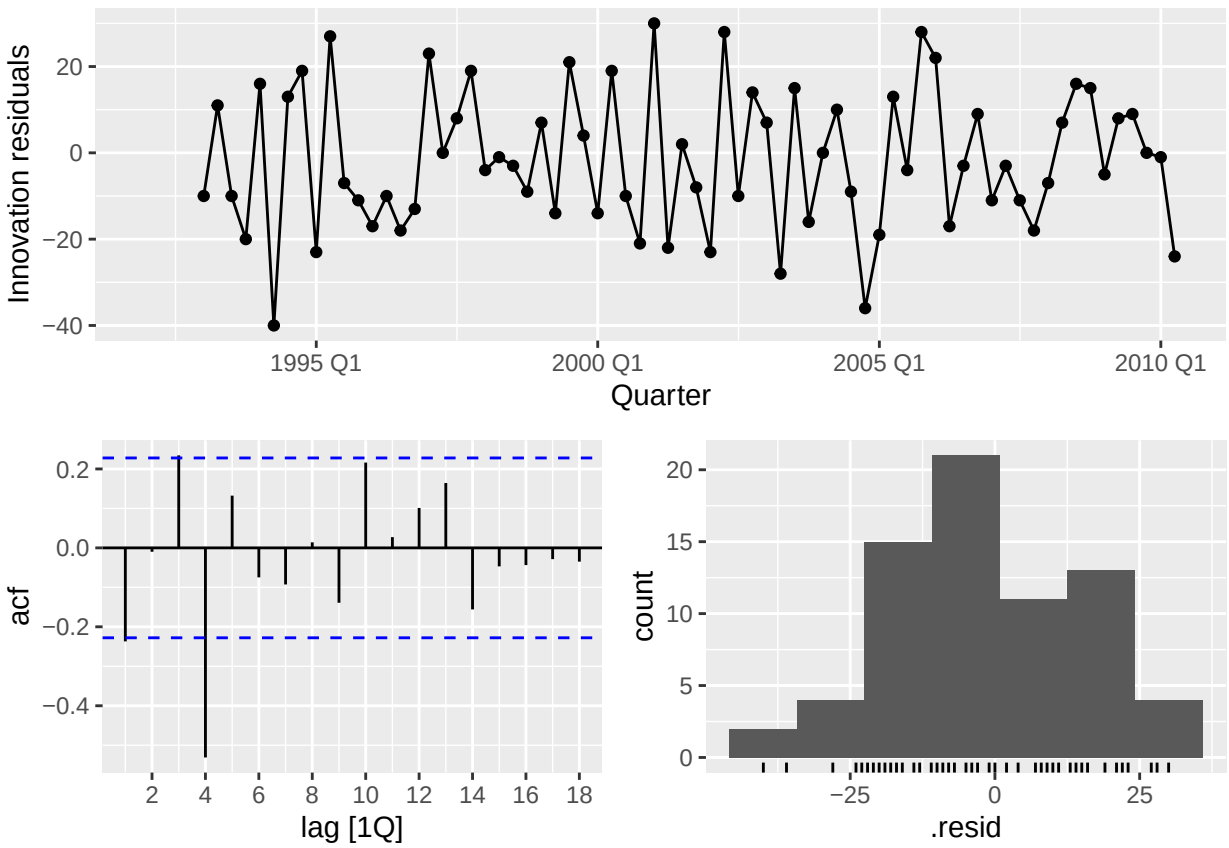
What do you conclude?

```r
# Extract data of interest
recent_production <- aus_production |>
  filter(year(Quarter) >= 1992)
fit <- recent_production |> model(SNAIVE(Beer))

# Generate the residual plots
residual_plots <- fit |> gg_tsresiduals()
```

```
## Warning: `gg_tsresiduals()` was deprecated in feasts 0.4.2.
## i Please use `ggtime::gg_tsresiduals()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

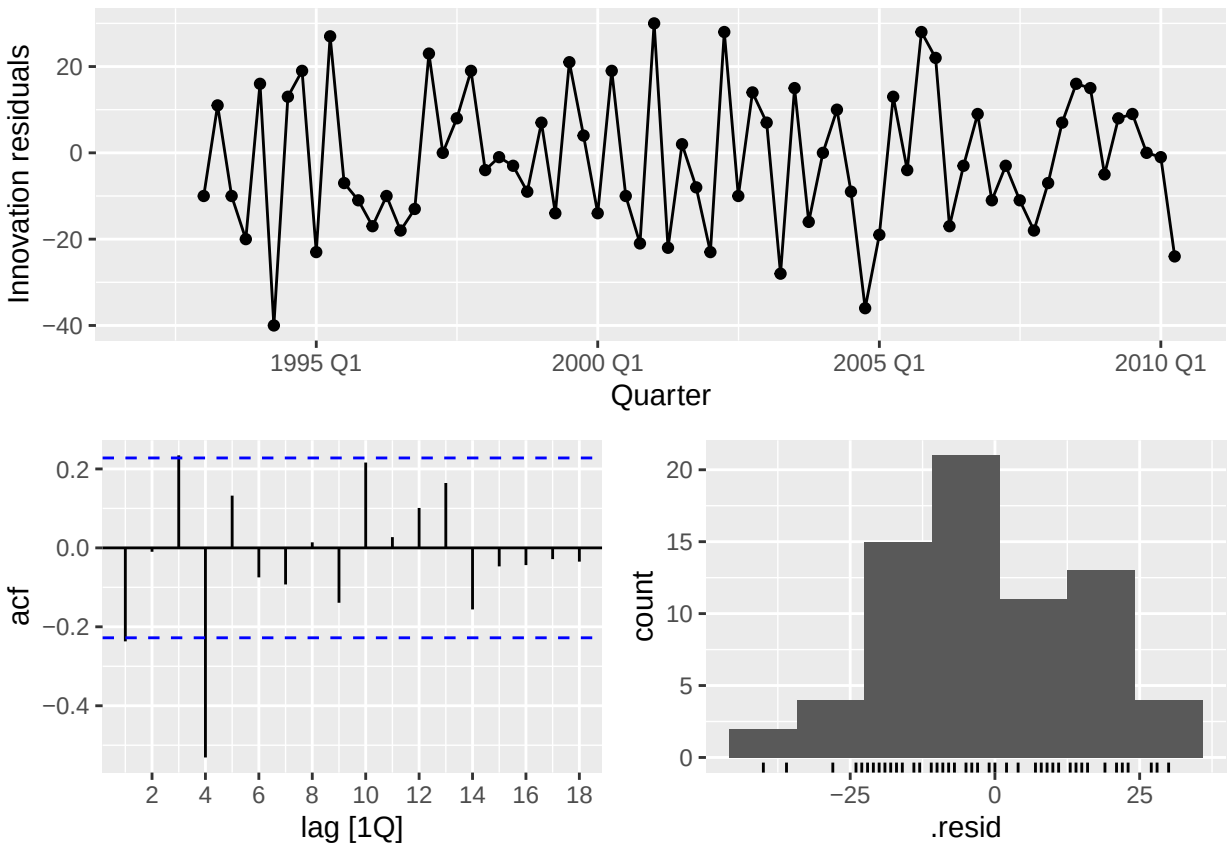
```r
print(residual_plots)
```

```
## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_line()`).

## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_point()`).

## Warning: Removed 4 rows containing non-finite outside the scale range
## (`stat_bin()`).

## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_rug()`).
```

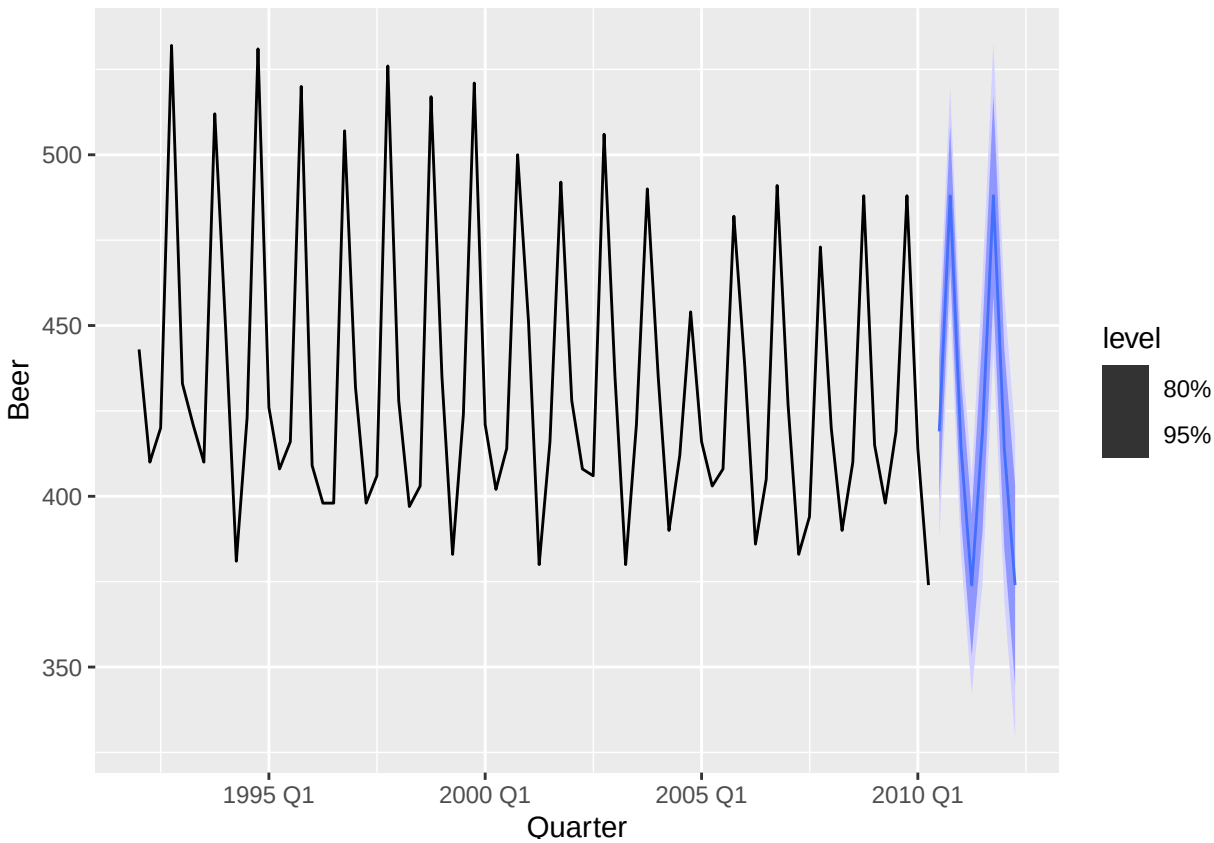



```r
# Extract data of interest
recent_production <- aus_production |>
  filter(year(Quarter) >= 1992)
# Define and estimate a model
fit <- recent_production |> model(SNAIVE(Beer))
# Look at the residuals
fit |> gg_tsresiduals()
```

```
## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_line()`).

## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_point()`).

## Warning: Removed 4 rows containing non-finite outside the scale range
## (`stat_bin()`).

## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_rug()`).
```




```
# Look a some forecasts
fit |> forecast() |> autoplot(recent_production)
```



RESIDUAL ANALYSIS FOR SEASONAL NAÏVE MODEL (BEER PRODUCTION)

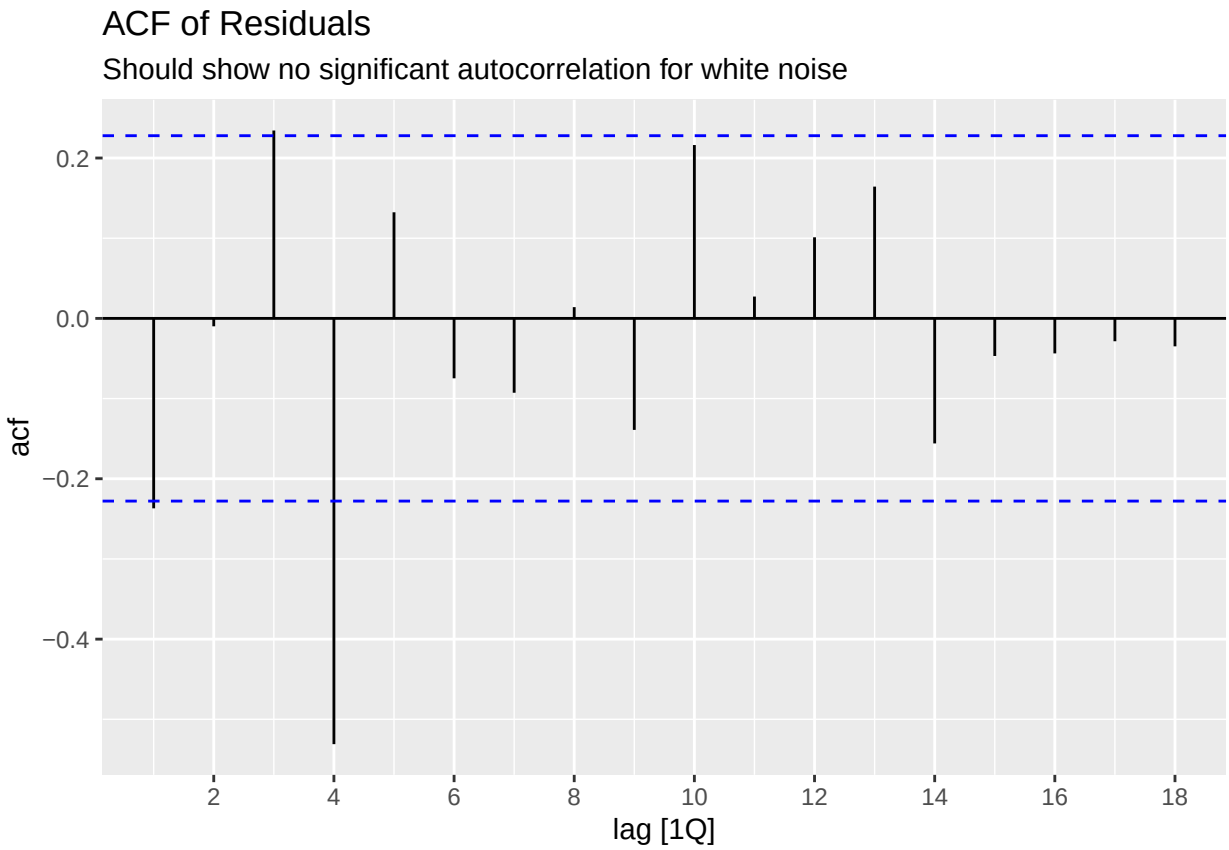
1. TIME PLOT OF RESIDUALS: The residual plot reveals clear seasonal patterns, with values alternating above and below zero in a systematic way. This cyclical behavior indicates that the model has not fully captured the seasonal structure in the data.
2. ACF PLOT: The autocorrelation function shows 4 lags with bars extending beyond the blue dashed confidence bands, with an additional 2 bars (lags 1 and 3) touching the boundary. This is substantially more than the 0-1 significant lags expected by random chance, confirming significant autocorrelation in the residuals.
3. HISTOGRAM: The residual distribution ranges from approximately -25 to +25, with a few values extending below -25 suggesting mild left skewness. However, the deviation from normality is not severe.

CONCLUSION:

- The residuals are NOT white noise. Despite being roughly normally distributed, they exhibit significant autocorrelation (confirmed by both the ACF plot).
- The seasonal patterns visible in the time plot and the multiple significant ACF bars indicate that the seasonal naïve model has failed to capture all the structure in the beer production data.
- A more sophisticated model (e.g., ETS or ARIMA with trend components) would likely produce better forecasts with white noise residuals.

```
# Get the residuals
residuals_df <- fit |> augment()

# Plot ACF of residuals
residuals_df %>%
  ACF(.resid) %>%
```

```r
  autoplot() +
  labs(title = "ACF of Residuals",
       subtitle = "Should show no significant autocorrelation for white noise")
```




```r
# Ljung-Box test for white noise
lb_test <- residuals_df %>%
  features(.resid, ljung_box, lag = 8)

print("Ljung-Box test results:")
```

```
## [1] "Ljung-Box test results:"
```

```r
print(lb_test)
```

```
## # A tibble: 1 x 3
##   .model       lb_stat lb_pvalue
##   <chr>          <dbl>     <dbl>
## 1 SNAIVE(Beer)    32.3 0.0000834
```

```r
# Interpret the p-value
p_value <- lb_test$lb_pvalue
cat("\nLjung-Box p-value:", round(p_value, 4))
```

```
##
## Ljung-Box p-value: 1e-04
```

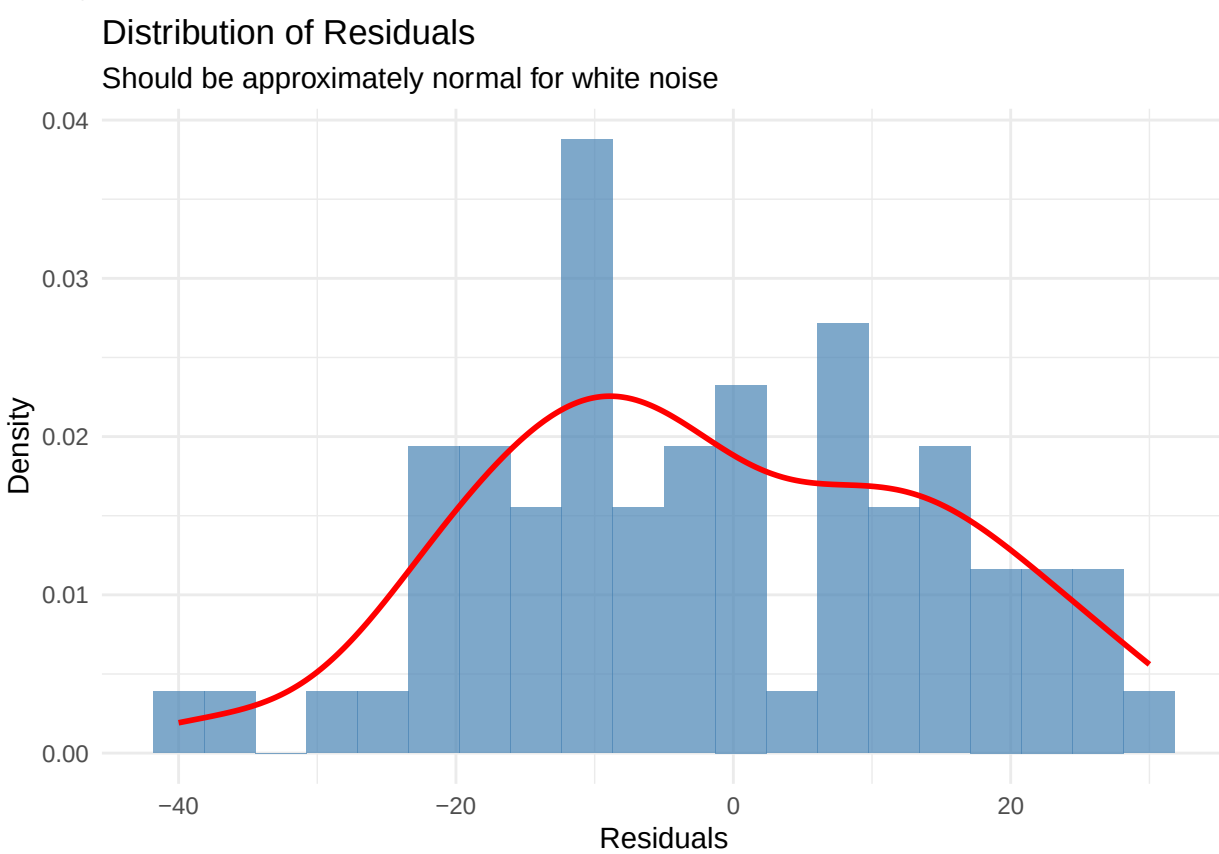
```r
if(p_value > 0.05) {
  cat("\n p-value > 0.05: Residuals appear to be white noise (no significant autocorrelation)")
} else {
```

```r
  cat("\n p-value < 0.05: Residuals show significant autocorrelation (not white noise)")
}
```

```
##
##  p-value < 0.05: Residuals show significant autocorrelation (not white noise)
```

```r
# Check distribution of residuals
residuals_df %>%
  ggplot(aes(x = .resid)) +
  geom_histogram(aes(y = after_stat(density)), bins = 20, fill = "steelblue", alpha = 0.7) +
  geom_density(color = "red", size = 1) +
  stat_function(fun = dnorm, args = list(mean = 0, sd = sd(residuals_df$.resid)),
                color = "darkgreen", linetype = "dashed", size = 1) +
  labs(title = "Distribution of Residuals",
       subtitle = "Should be approximately normal for white noise",
       x = "Residuals", y = "Density") +
  theme_minimal()
```

```
## Warning: Removed 4 rows containing non-finite outside the scale range
## (`stat_bin()`).

## Warning: Removed 4 rows containing non-finite outside the scale range
## (`stat_density()`).

## Warning: Removed 101 rows containing missing values or values outside the scale range
## (`geom_function()`).
```



Key Findings: The Ljung-Box test returns a p-value of 0.0001, which is well below the 0.05 significance threshold.

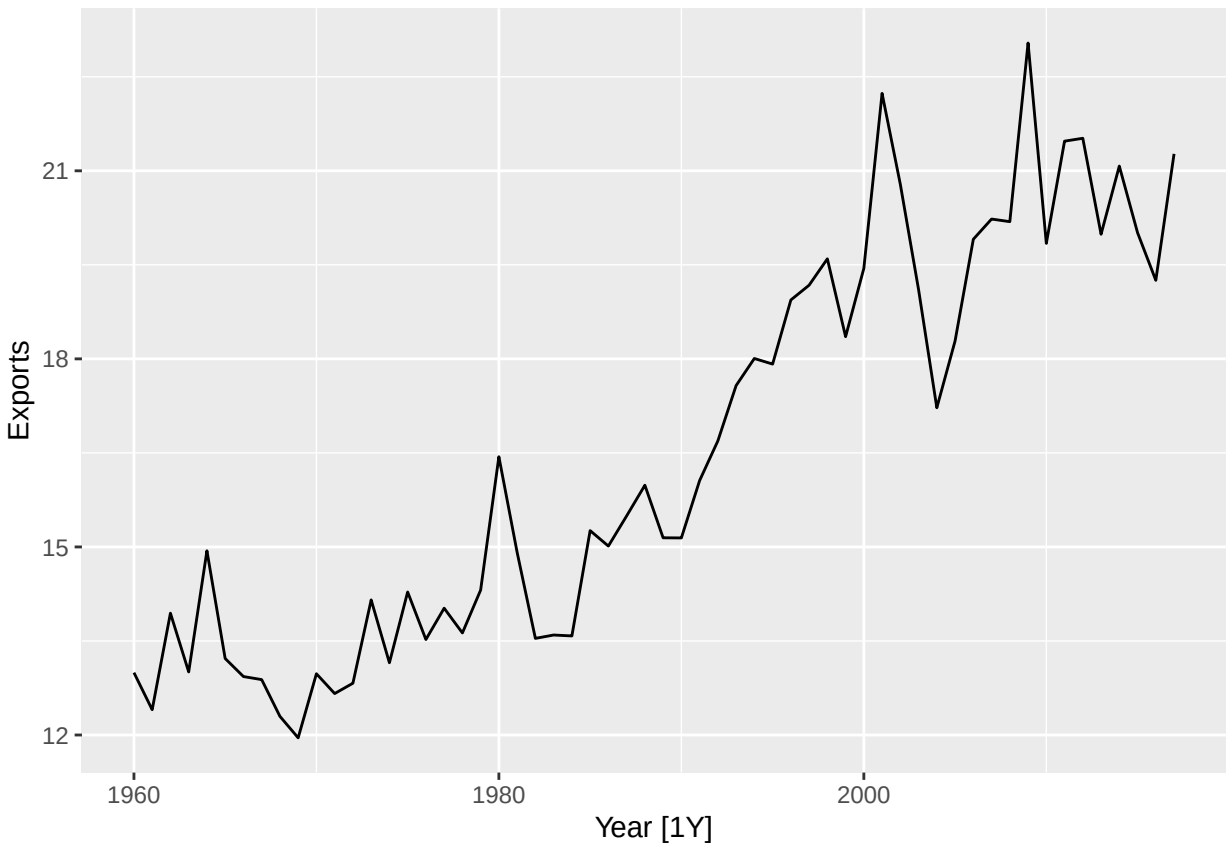
This indicates significant autocorrelation remains in the residuals, meaning they are NOT white noise. Therefore, the seasonal naïve model is inadequate for this beer production data.

5.4 Repeat the previous exercise using the Australian Exports series from global_economy and the Bricks series from aus_production. Use whichever of NAIVE() or SNAIVE() is more appropriate in each case.

```
# Get Australian exports data
aus_exports <- global_economy %>%
  filter(Country == "Australia") %>%
  select(Year, Exports)

# Plot the data
aus_exports %>%
  autoplot(Exports) +
  labs(title = "Australian Exports (% of GDP)", y = "Exports")
```

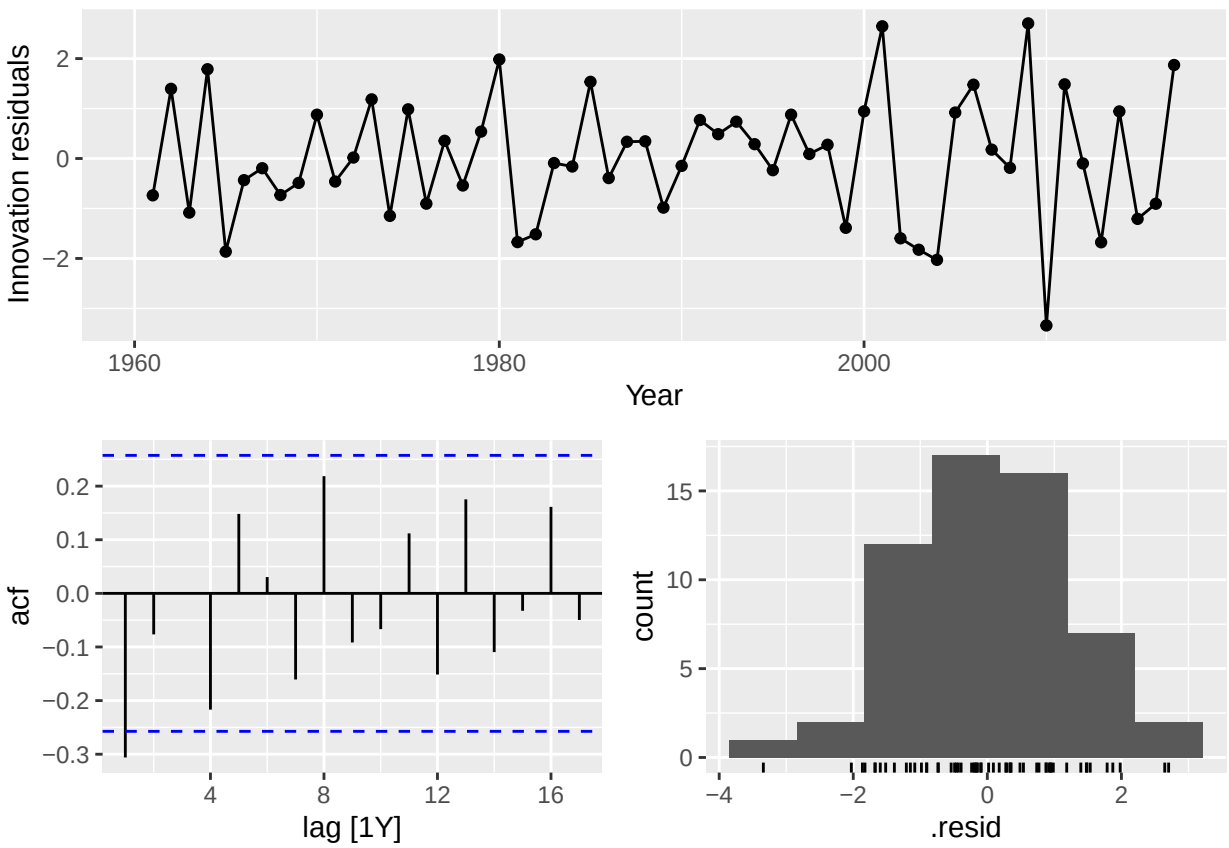
Australian Exports series from global_economy

```
# Check for seasonality (annual data - no seasonality)
# Use NAIVE or RW with drift?
aus_exports %>%
  autoplot(Exports)  # Shows trend, no seasonality
```




```r
# Fit appropriate model (NAIVE with drift for trending annual data)
fit_exports <- aus_exports %>%
  model(Drift = RW(Exports ~ drift()))

# Check residuals
fit_exports %>% gg_tsresiduals()
```

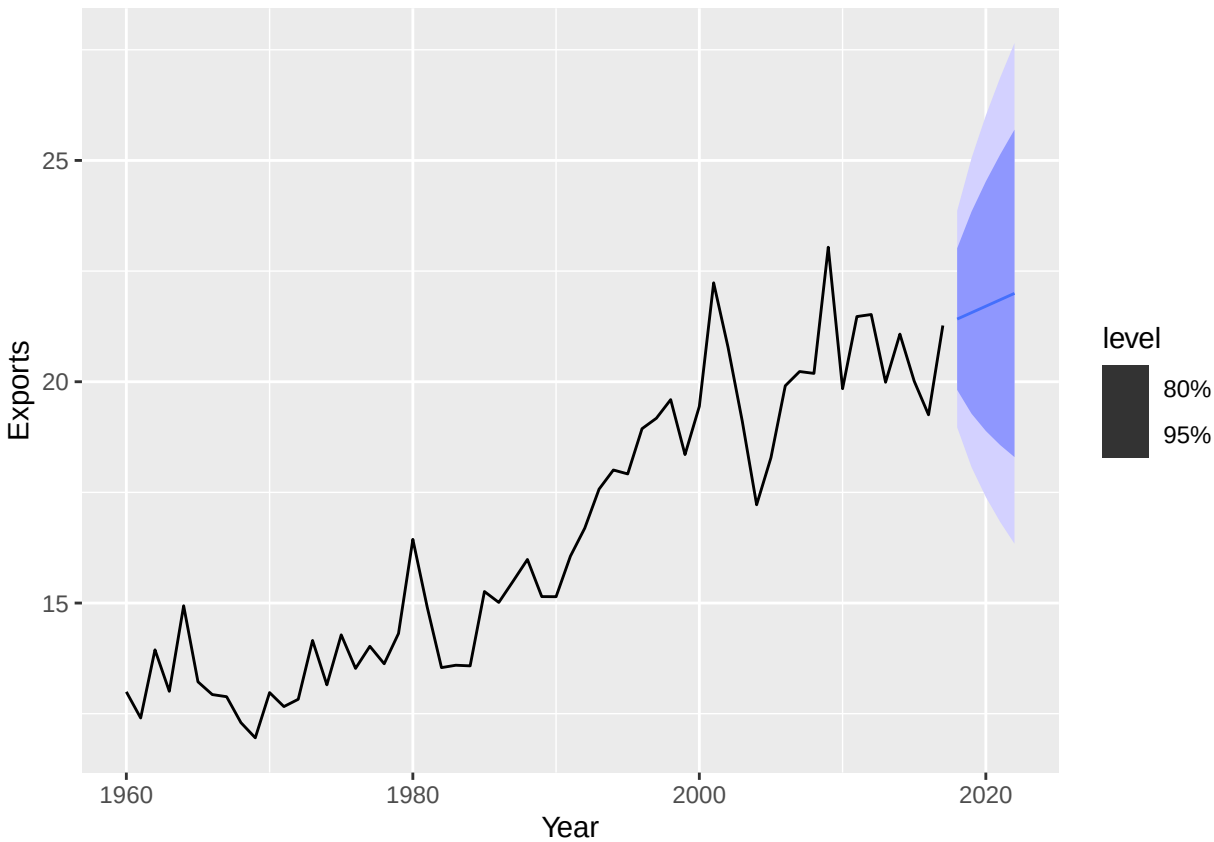
```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).

## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_bin()`).

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_rug()`).
```




```r
# Ljung-Box test
fit_exports %>% augment() %>% features(.resid, ljung_box, lag = 10)
```

```
## # A tibble: 1 x 3
##   .model lb_stat lb_pvalue
##   <chr>    <dbl>     <dbl>
## 1 Drift     16.4    0.0896
```

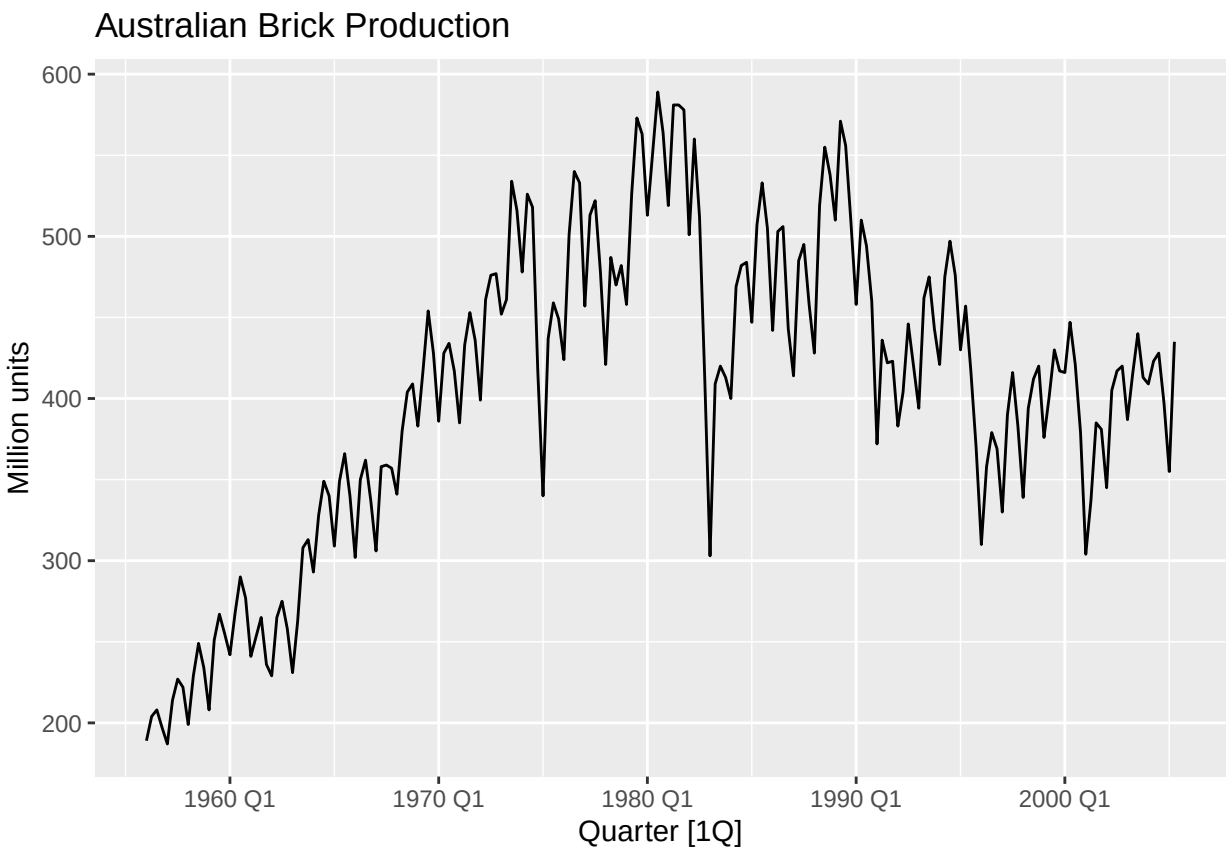
```r
# Generate forecasts
fit_exports %>% forecast(h = 5) %>% autoplot(aus_exports)
```



Key Findings:

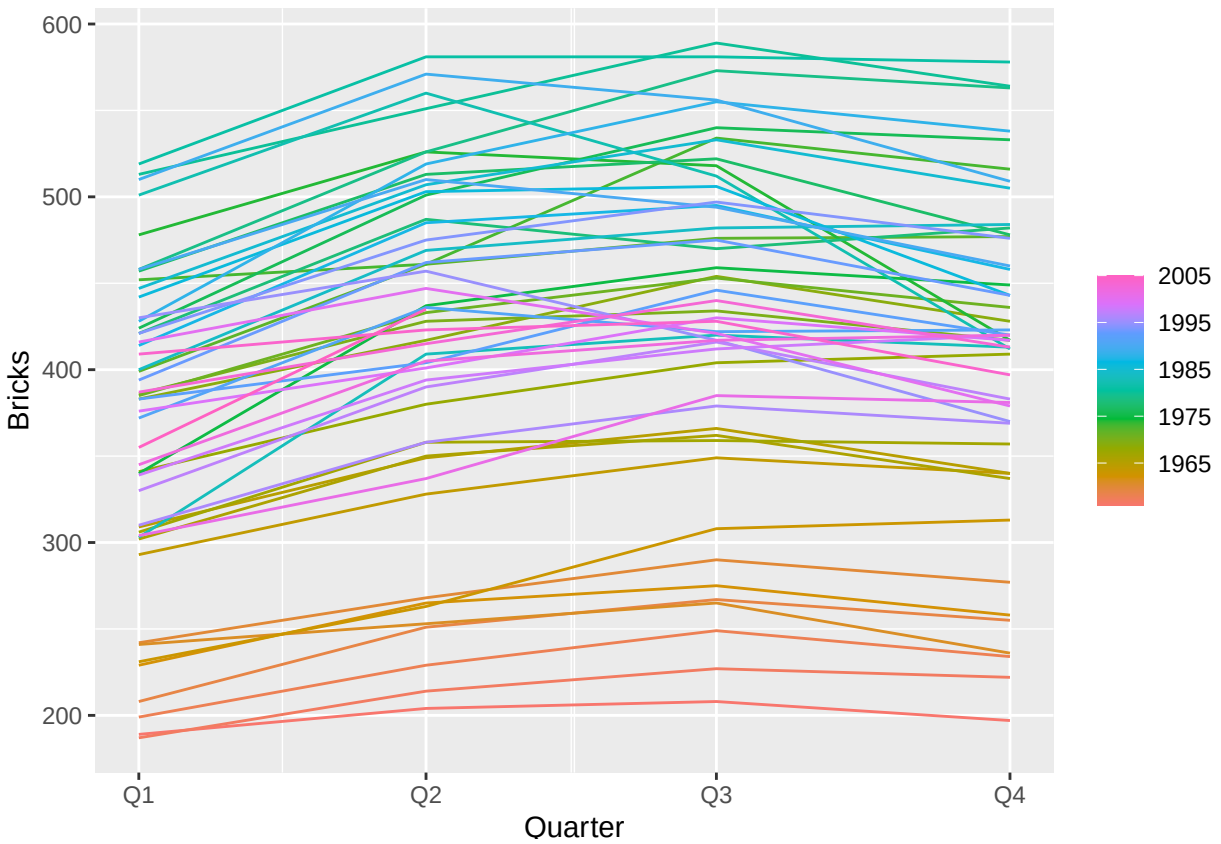
- The residual time plot shows some seasonal patterns.
- ACF has only one lag slightly outside the confidence bands.
- The residuals are normally distributed (range -2 to 2) Ljung-Box test ($p = 0.09 > 0.05$) confirms no significant autocorrelation, indicating the drift model is adequate for Australian exports.

```
# Get bricks data (quarterly)
bricks <- aus_production %>%
  select(Quarter, Bricks) %>%
  filter(!is.na(Bricks))

# Plot the data
bricks %>%
  autoplot(Bricks) +
  labs(title = "Australian Brick Production", y = "Million units")
```



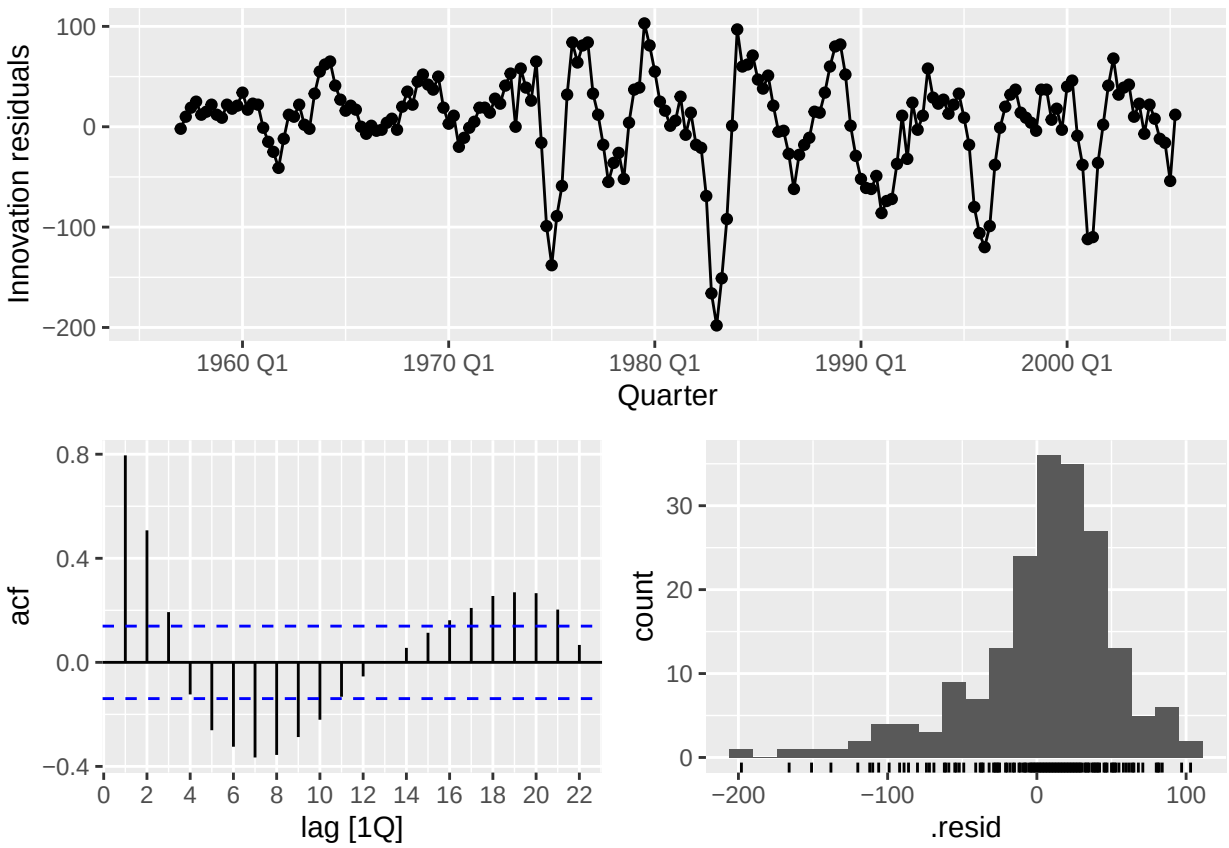
Bricks data

```
# Check for seasonality
bricks %>% gg_season(Bricks)  # Clear quarterly pattern
```




```r
# Fit SNAIVE (appropriate for quarterly seasonal data)
fit_bricks <- bricks %>%
  model(SNAIVE = SNAIVE(Bricks))

# Check residuals
fit_bricks %>% gg_tsresiduals()
```

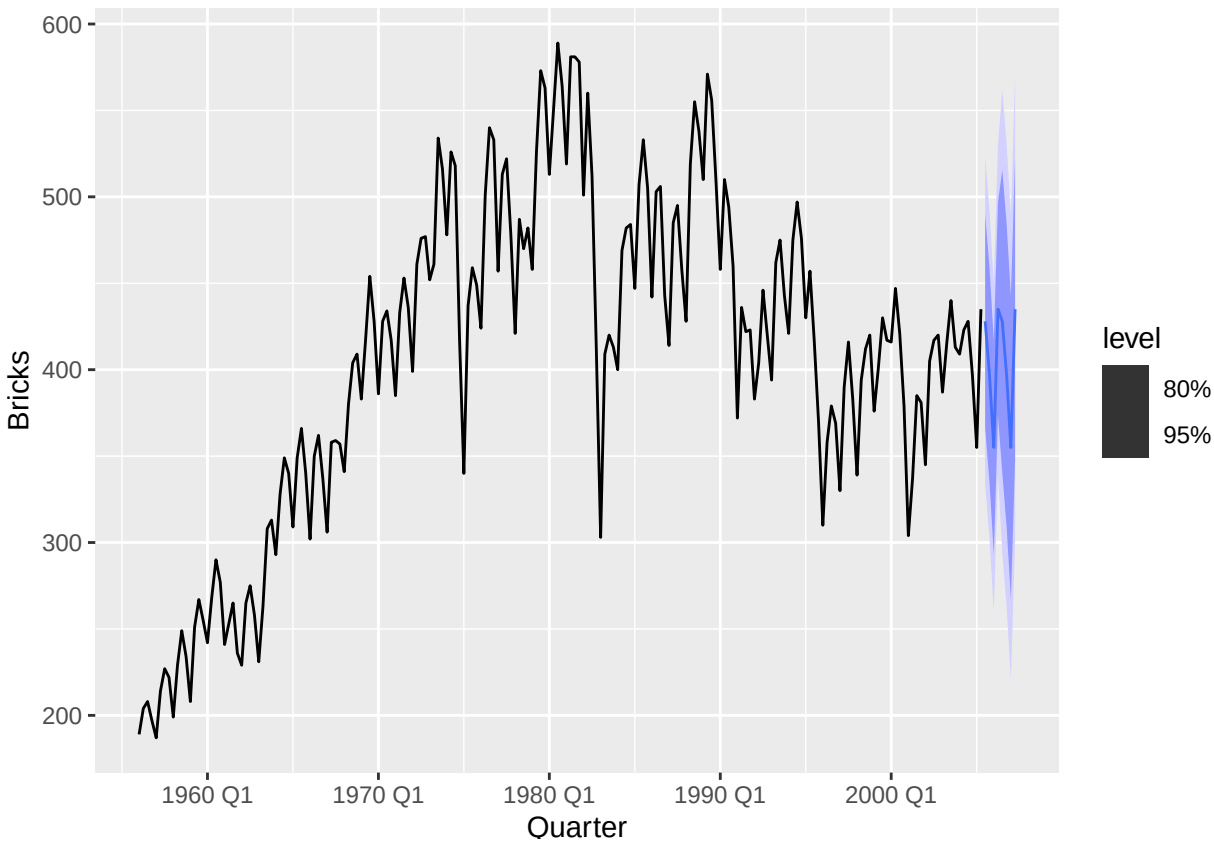
```
## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_line()`).

## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_point()`).

## Warning: Removed 4 rows containing non-finite outside the scale range
## (`stat_bin()`).

## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_rug()`).
```




```r
# Ljung-Box test
fit_bricks %>% augment() %>% features(.resid, ljung_box, lag = 8)
```

```
## # A tibble: 1 x 3
##   .model lb_stat lb_pvalue
##   <chr>    <dbl>     <dbl>
## 1 SNAIVE    274.         0
```

```r
# Generate forecasts
fit_bricks %>% forecast(h = 8) %>% autoplot(bricks)
```



Key Findings:

- The residuals show clear problems: the Ljung-Box test (p = 0) confirms significant autocorrelation, with many ACF lags outside the confidence bands.
- The residual histogram ranges from -100 to +100 with extreme outliers near -200, indicating the SNAIVE model is inadequate for brick production data.

Conclusions:

1. NAIVE with drift is appropriate for Australian exports as it shows annual data with trend but no seasonality. The Ljung-Box test (p = 0.09) confirms residuals are white noise, indicating the model adequately captures the patterns.
2. SNAIVE is appropriate for bricks data due to strong quarterly seasonality. However, the residuals show significant autocorrelation (p = 0) and outliers, suggesting the model misses some patterns despite capturing the seasonal component.

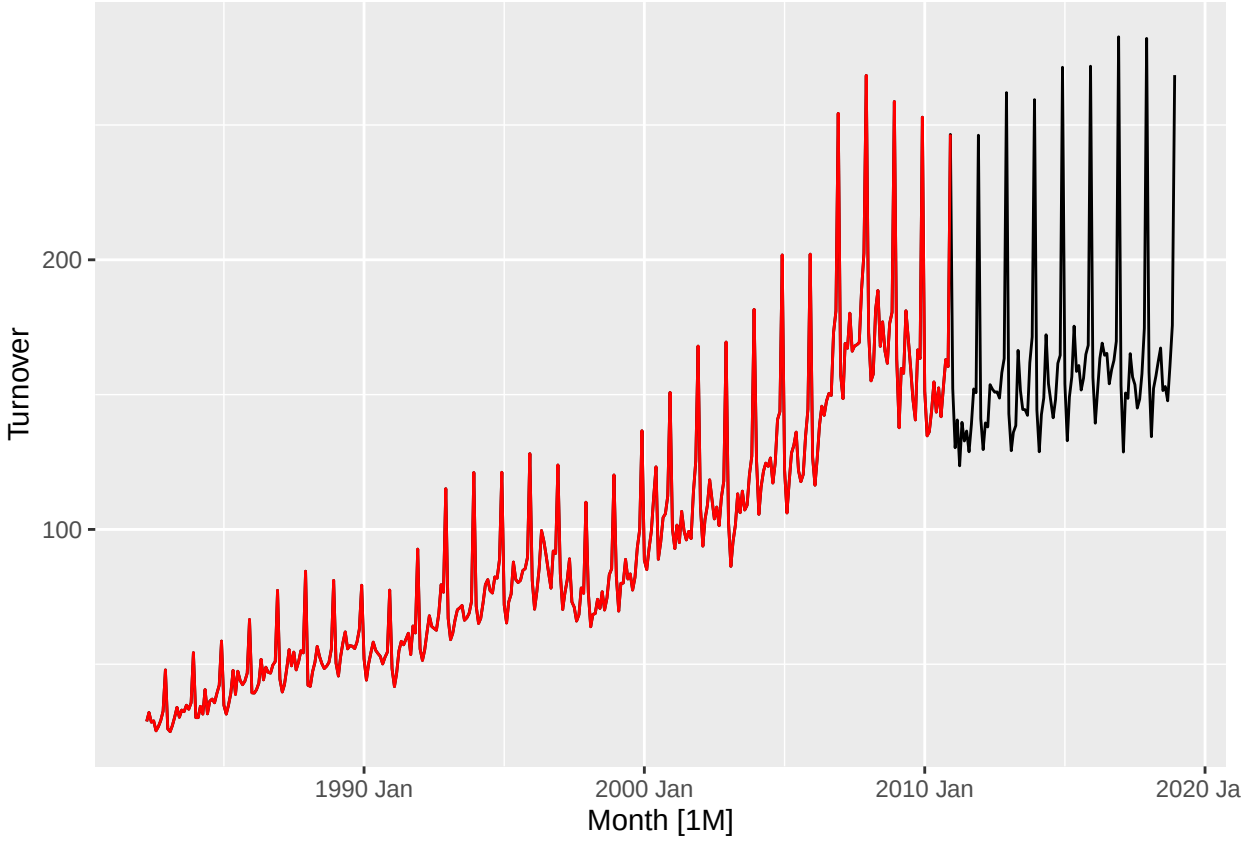
5.7 For your retail time series (from Exercise 7 in Section 2.10):

```
set.seed(12345)
myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`,1))
myseries_train <- myseries |>
  filter(year(Month) < 2011)
```

a. Create a training dataset consisting of observations before 2011 using

```r
autoplot(myseries, Turnover) +
  autolayer(myseries_train, Turnover, colour = "red")
```

b. Check that your data have been split appropriately by producing the following plot.



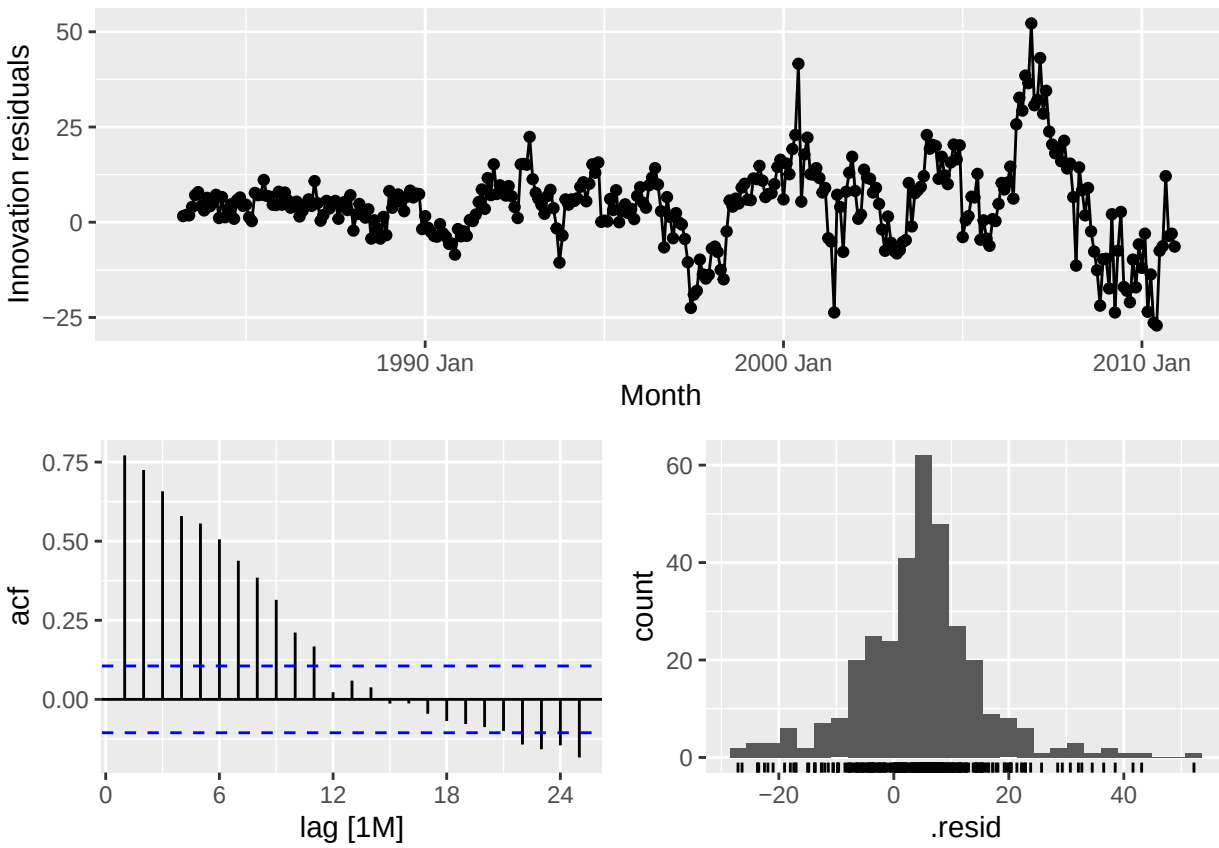

```r
fit <- myseries_train |>
  model(snaive = SNAIVE(Turnover))
```

c. Fit a seasonal naïve model using SNAIVE() applied to your training data (myseries_train).

```r
fit |> gg_tsresiduals()
```

d. Check the residuals.

```
## Warning: Removed 12 rows containing missing values or values outside the scale range
## (`geom_line()`).

## Warning: Removed 12 rows containing missing values or values outside the scale range
## (`geom_point()`).

## Warning: Removed 12 rows containing non-finite outside the scale range
## (`stat_bin()`).

## Warning: Removed 12 rows containing missing values or values outside the scale range
## (`geom_rug()`).
```



Do the residuals appear to be uncorrelated and normally distributed? ***No, the residuals do NOT appear to be uncorrelated and normally distributed.***

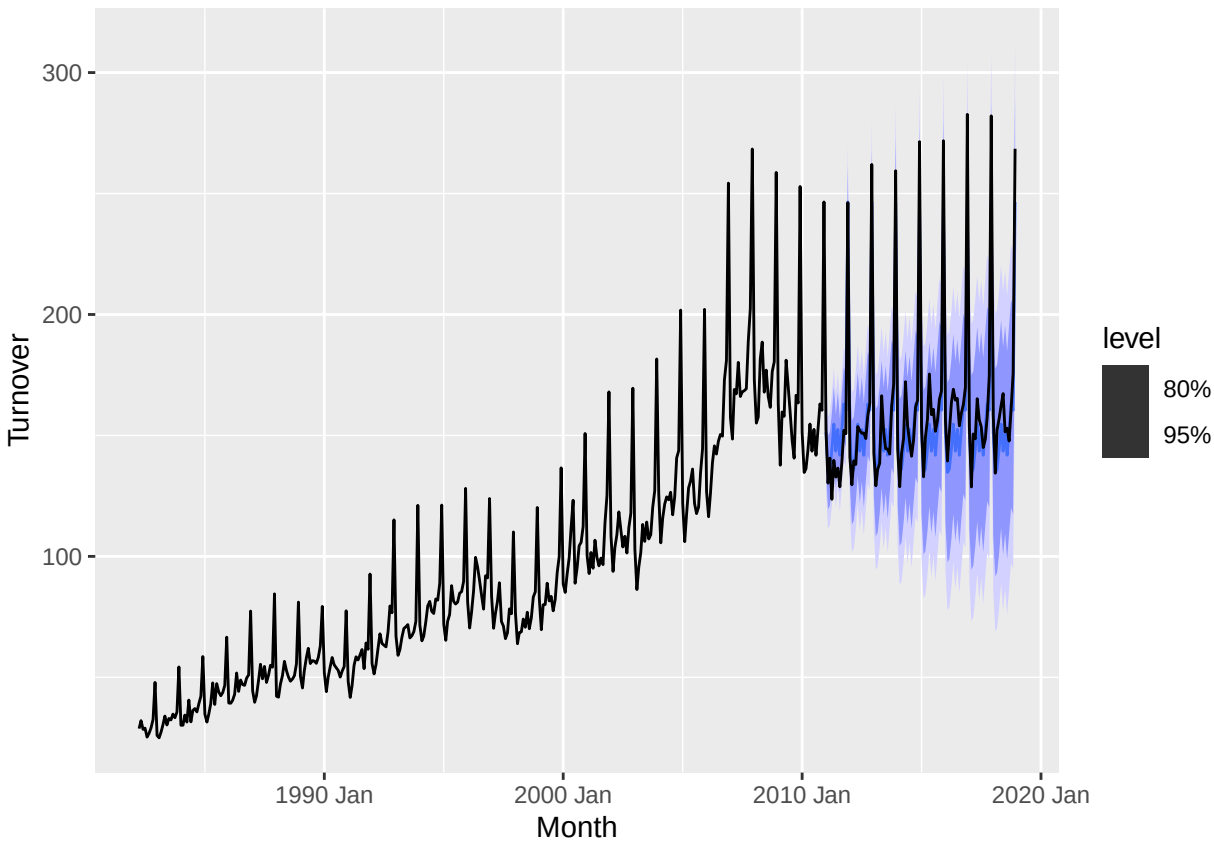
- The ACF plot shows many lines outside the dashed confidence bands, indicating significant autocorrelation (correlated residuals).
- While the histogram is roughly normal between -20 and +20, the presence of outliers at 40 and 50 suggests departures from normality.
- The residual time plot also shows cyclical patterns, confirming the model has failed to capture all structure in the data.

```
fc <- fit |>
  forecast(new_data = anti_join(myseries, myseries_train))
```

e. Produce forecasts for the test data

```
## Joining with `by = join_by(State, Industry, `Series ID`, Month, Turnover)`
```

```
fc |> autoplot(myseries)
```




```r
# Simple but clean table
accuracy_table <- bind_rows(
  Training = fit |> accuracy(),
  Test = fc |> accuracy(myseries),
  .id = "Dataset"
) %>%
  select(Dataset, RMSE, MAE, MAPE, MASE) %>%
  mutate(across(where(is.numeric), ~round(., 2)))

print(accuracy_table)
```

f. Compare the accuracy of your forecasts against the actual values.

```
## # A tibble: 2 x 5
##   Dataset   RMSE   MAE  MAPE  MASE
##   <chr>    <dbl> <dbl> <dbl> <dbl>
## 1 Training  11.8  8.93  9.64  1
## 2 Test      11.8  9.25  5.55  1.04
```

```r
# Or with knitr for better formatting

kable(accuracy_table, caption = "Training vs Test Accuracy", digits = 2)
```

Table 1: Training vs Test Accuracy

Dataset	RMSE	MAE	MAPE	MASE
Training	11.85	8.93	9.64	1.00
Test	11.76	9.25	5.55	1.04

g. How sensitive are the accuracy measures to the amount of training data used? The accuracy measures show relatively low sensitivity to the training data amount in this case.

- The test set performance (RMSE = 11.76, MAE = 9.25) is very close to the training set performance (RMSE = 11.85, MAE = 8.93), indicating the model generalizes well.
- Interestingly, the MAPE improved from 9.64% on training to 5.55% on test, suggesting the test period may have been more stable or predictable.
- The MASE of 1.04 on test data confirms the model performs nearly as well as a naive method. Overall, the model appears robust to the training data size, with no evidence of severe overfitting or underfitting.